

SQL মাস্টারি

কনসেপ্ট থেকে টপ-ক্লাস দক্ষতা পর্যন্ত — সম্পূর্ণ গাইড

২০+ কনসেপ্ট অধ্যায় • ১২০+ সমাধানসহ অনুশীলন • নিউজ প্ল্যাটফর্ম প্রজেক্টের বাস্তব টেবিল দিয়ে

Part A — কনসেপ্ট (২০ অধ্যায়)

Part B — অনুশীলন (১০ স্তর, সহজ থেকে এক্সপার্ট পর্যন্ত)

এই নোট কীভাবে ব্যবহার করবে

এই নোট দুই ভাগে গঠিত — **Part A**-তে SQL-এর প্রতিটি গুরুত্বপূর্ণ কনসেপ্ট গভীরভাবে ব্যাখ্যা করা আছে, প্রতিটির সাথে এই নিউজ প্ল্যাটফর্ম প্রজেক্টের আসল টেবিল দিয়ে উদাহরণ। **Part B**-তে ১২০+টা অনুশীলন প্রশ্ন আছে, কঠিনতা অনুযায়ী স্তরে ভাগ করা (সহজ → মাঝারি → কঠিন → এক্সপার্ট), প্রতিটির সম্পূর্ণ সমাধান ও ব্যাখ্যাসহ।

কীভাবে পড়লে সবচেয়ে বেশি উপকার পাবে

- প্রথমে **Part A**-এর একটা অধ্যায় পড়ো, বুঝে নাও কনসেপ্টটা কেন দরকার।
- তারপর **Part B** থেকে সেই কনসেপ্ট-সম্পর্কিত অনুশীলনগুলোতে যাও (নিচে ম্যাপিং টেবিল আছে) — সমাধান না দেখে নিজে চেষ্টা করো।
- আটকে গেলে ইঙ্গিত পড়ো, তাতেও না হলে সমাধান দেখো — কিন্তু সমাধান দেখার পর সেটা বন্ধ করে আবার নিজে থেকে লিখে দেখো।
- সম্ভব হলে একটা লোকাল **MySQL/SQLite**-এ এই প্রজেক্টের স্কিমা বানিয়ে সত্যিকারের ডেটা দিয়ে প্রতিটা কুয়েরি রান করে দেখো — শুধু পড়ে বোঝা আর নিজে হাতে চালিয়ে দেখা সম্পূর্ণ আলাদা স্তরের শেখা।

Part A অধ্যায়-তালিকা

অধ্যায়	বিষয়
১	ডেটা টাইপ ও স্কিমা ডিজাইনের বেসিক
২	SELECT, WHERE ও ফিল্টারিং অপারেটর
৩	ORDER BY, LIMIT/OFFSET, DISTINCT
৪	JOIN — সব ধরনের
৫	Self-JOIN ও Hierarchical ডেটা
৬	Aggregation — COUNT, SUM, AVG, GROUP BY
৭	HAVING — গ্রুপ ফিল্টারিং গভীরভাবে
৮	Subquery — Scalar, IN, EXISTS, Correlated
৯	CTE (Common Table Expression)
১০	Window Functions
১১	Set Operations — UNION, INTERSECT, EXCEPT
১২	CASE WHEN ও Conditional Logic
১৩	String Functions
১৪	Date/Time Functions
১৫	NULL হ্যান্ডলিং
১৬	Index ও Query Planner
১৭	N+1 সমস্যা ও Eager Loading
১৮	Full-text Search ও JSON কুয়েরি
১৯	Partitioning ও Partition Pruning
২০	Pagination — Offset vs Cursor
২১	Transaction, Locking ও Isolation Levels
২২	Normalization ও Denormalization
২৩	View, Stored Procedure, Trigger — পরিচিতি

মডিউল ১

ডেটা টাইপ ও স্কিমা ডিজাইনের বেসিক

সঠিক টাইপ বেছে নেওয়া কেন গুরুত্বপূর্ণ

সংখ্যা টাইপ

টাইপ	রেঞ্জ/ব্যবহার	এই প্রজেক্টে উদাহরণ
TINYINT	-128 থেকে 127 (বা UNSIGNED 0-255)	priority level, satisfaction_rating (1-5)
SMALLINT	-32,768 থেকে 32,767	volume, issue_number, viewport_width
INT	-২১৪ কোটি থেকে ২১৪ কোটি	batch (migrations), sort priorities
BIGINT	বিশাল রেঞ্জ, প্রায় সব PK/FK-এর জন্য	সব primary key: id BIGINT UNSIGNED AUTO_INCREMENT
DECIMAL(p,s)	নির্ভুল দশমিক সংখ্যা (টাকা-পয়সার জন্য বাধ্যতামূলক)	price, amount, mrr_amount, rate_amount
FLOAT/DOUBLE	আনুমানিক দশমিক — টাকার হিসাবে কখনো ব্যবহার করো না	এই প্রজেক্টে ব্যবহৃত হয়নি টাকার জন্য

কেন DECIMAL, FLOAT না: FLOAT/DOUBLE বাইনারিতে সংরক্ষিত হয়, তাই $0.1 + 0.2$ প্রায়ই 0.30000000000000004 হয়ে যায় — সামান্য রাউন্ডিং ভুল টাকার হিসাবে জমতে জমতে বড় সমস্যা তৈরি করে। DECIMAL সংখ্যাকে স্ট্রিং-এর মতো নির্ভুলভাবে সংরক্ষণ করে, তাই সব financial কলাম (amount, price, rate, mrr) সবসময় DECIMAL(10,2) টাইপের।

টেক্সট টাইপ

টাইপ	ব্যবহার	এই প্রজেক্টে উদাহরণ
VARCHAR(n)	ছোট থেকে মাঝারি টেক্সট, সর্বোচ্চ দৈর্ঘ্য জানা থাকলে	slug VARCHAR(255), email VARCHAR(191)
TEXT	বড় টেক্সট, দৈর্ঘ্য অনির্দিষ্ট	body (comments), description
LONGTEXT	অত্যন্ত বড় টেক্সট	payload (jobs), exception (failed_jobs)
ENUM	নির্দিষ্ট কয়েকটা মানের একটা — একটা mini-lookup ইনলাইন	status, priority, billing_cycle
JSON	নমনীয় স্ট্রাকচার ডেটা	features, geo_restriction, condition_logic

email VARCHAR(191) কেন 255 না: MySQL-এর utf8mb4 charset-এ InnoDB-এর একটা ইনডেক্স ৭৬৭ বাইট পর্যন্ত সীমাবদ্ধ (পুরনো MySQL ভার্সনে) — utf8mb4-এ প্রতি ক্যারেক্টার ৪ বাইট পর্যন্ত নিতে পারে, তাই $767 \div 4 \approx 191$ । এই ঐতিহাসিক সীমাবদ্ধতার কারণে ইউনিক ইনডেক্স করা VARCHAR কলাম প্রায়ই 191-এ সীমাবদ্ধ রাখা হয়।

সময় টাইপ

টাইপ	পার্শ্বক্য
DATE	শুধু তারিখ (YYYY-MM-DD) — publication_date, birth_date
TIMESTAMP	তারিখ+সময়, টাইমজোন-সচেতন, রেঞ্জ সীমিত (1970-2038) — created_at, published_at
DATETIME	তারিখ+সময়, টাইমজোন-অসচেতন, রেঞ্জ বড় — partition key কলামে (read_at, recorded_at) বাধ্যতামূলক

মডিউল-১০-এ (behavioral_metrics, read_histories) recorded_at/read_at কলাম DATETIME, TIMESTAMP না — কারণ MySQL partitioning শুধু DATETIME/DATE-ভিত্তিক এক্সপ্রেশনে কাজ করে, TIMESTAMP-এ না।

Constraint — ডেটা সঠিক রাখার গার্ডরেল

- NOT NULL — কলাম কখনো খালি থাকতে পারবে না (headline, email)।
- UNIQUE — একই মান দুইবার থাকতে পারবে না (slug, email, transaction_id)।
- DEFAULT — মান না দিলে স্বয়ংক্রিয় ডিফল্ট বসে (status DEFAULT 'draft')।
- FOREIGN KEY — অন্য টেবিলের বৈধ ID ছাড়া রেফারেন্স করা যাবে না (referential integrity)।
- CHECK (MySQL 8.0.16+) — কাস্টম শর্ত, যেমন CHECK (price >= 0)।
- প্রশ্ন: কেন DECIMAL(10,2) লিখলে সর্বোচ্চ কত টাকা পর্যন্ত সংরক্ষণ করা যাবে (হিট: ১০ সংখ্যা মোট, ২টা দশমিকের পরে)?

মডিউল ২

SELECT, WHERE ও ফিল্টারিং অপারেটর

ডেটা বাছাইয়ের প্রতিটা টুল

তুলনামূলক অপারেটর

অপারেটর	অর্থ	উদাহরণ
=, !=/<>	সমান, অসমান	WHERE status = 'published'
>, <, >=, <=	তুলনা	WHERE view_count > 1000
BETWEEN a AND b	সীমার মধ্যে (ইনক্লুসিভ)	WHERE published_at BETWEEN '2026-01-01' AND '2026-01-31'
IN (...)	তালিকার যেকোনো একটি	WHERE status IN ('draft', 'pitch', 'in_review')
LIKE	প্যাটার্ন ম্যাচিং (% = যেকোনো, _ = একটি ক্যারেক্টার)	WHERE email LIKE '%@gmail.com'
IS NULL / IS NOT NULL	NULL চেক (= NULL কখনো কাজ করে না!)	WHERE deleted_at IS NULL

সবচেয়ে সাধারণ বিগিনার ভুল: **WHERE deleted_at = NULL** লেখা — এটা কখনো সত্য হয় না! **NULL** মানে "অজানা", আর **SQL**-এ কোনো কিছু "অজানা"-র সমান কিনা তা কখনো নিশ্চিত করে বলা যায় না। তাই সবসময় **IS NULL / IS NOT NULL** ব্যবহার করতে হয়।

লজিক্যাল অপারেটর — AND, OR, NOT

কোড	ব্যাখ্যা
SELECT * FROM articles WHERE status = 'published' AND (is_breaking = 1 OR is_developing_story = 1)	বন্ধনী ব্যবহার জরুরি — AND সবসময় OR -এর আগে evaluate হয় (operator precedence), তাই বন্ধনী ছাড়া লিখলে লজিক ভুল হয়ে যাবে

AND সবসময় **OR**-এর চেয়ে বেশি "শক্তিশালী" (precedence বেশি) — গণিতে **multiplication** যেমন **addition**-এর আগে হয়, ঠিক তেমন। **WHERE a=1 OR a=2 AND b=3** আসলে পড়া হয় **WHERE a=1 OR (a=2 AND b=3)** — এই কারণে জটিল কন্ডিশনে সবসময় নিজের হাতে বন্ধনী দেওয়া উচিত, **MySQL**-এর ডিফল্ট **precedence**-এর উপর ভরসা না করে।

EXISTS vs IN — কখন কোনটা

পদ্ধতি	কোড	পারফরম্যান্স আচরণ
IN	SELECT * FROM articles WHERE primary_category_id IN (SELECT id FROM categories WHERE is_sensitive=1)	ভেতরের subquery প্রথমে সম্পূর্ণ execute হয়ে একটি তালিকা তৈরি করে, তারপর বাইরের কুয়েরি প্রতিটা রো সেই তালিকার সাথে মেলায়
EXISTS	SELECT * FROM articles a WHERE EXISTS (SELECT 1 FROM categories c WHERE c.id = a.primary_category_id AND c.is_sensitive=1)	প্রতিটা বাইরের রো-এর জন্য ভেতরের কুয়েরি চেক করে শুধু "কোনো ম্যাচ আছে কিনা" (true/false) — প্রথম ম্যাচ পেলেই থেমে যায়, পুরো তালিকা তৈরি করে না

সাধারণ নিয়ম: **subquery**-র ফলাফল ছোট হলে **IN** দ্রুত, **subquery** বড়/জটিল হলে বা বাইরের টেবিল বড় হলে **EXISTS** প্রায়ই দ্রুত (কারণ **early-exit** করতে পারে)। আধুনিক **MySQL optimizer (8.0+)** প্রায়ই দুটোকে একই **execution plan**-এ রূপান্তর করে, তাই বাস্তবে পার্থক্য কম — কিন্তু **NULL** থাকলে বড় পার্থক্য হয়: **NOT IN** একটা **subquery**-তে যদি একটাও **NULL** থাকে পুরো কুয়েরি কোনো রেজাল্ট দেয় না (একটা কুখ্যাত ফাঁদ), **NOT EXISTS** এই সমস্যায় পড়ে না।

- প্রশ্ন: *WHERE x NOT IN (SELECT y FROM t) — যদি t.y কলামে একটা NULL থাকে, তাহলে এই কুয়েরি কী রিটার্ন করবে, আর কেন?*

ORDER BY, LIMIT/OFFSET, DISTINCT

ফলাফল সাজানো ও সীমিত করা

ORDER BY — একাধিক কলাম, ASC/DESC মিশ্রণ

কোড	ব্যাখ্যা
SELECT * FROM articles ORDER BY is_breaking DESC, published_at DESC	প্রথমে is_breaking অনুযায়ী সাজায় (breaking আগে), তারপর একই is_breaking মানের ভেতরে published_at অনুযায়ী — multi-column sort বাম থেকে ডানে অগ্রাধিকার পায়

ORDER BY-তে NULL সবসময় সবচেয়ে ছোট মান হিসেবে গণ্য হয় MySQL-এ — ASC হলে সবার আগে, DESC হলে সবার শেষে আসে। কোনো নির্দিষ্ট কলামে NULL-কে জোর করে শেষে শুরুতে রাখতে চাইলে: ORDER BY (column IS NULL), column ব্যবহার করা যায় (IS NULL ফলস/ট্রু কে 0/1 হিসেবে গণনা করে)।

LIMIT/OFFSET — মৌলিক ব্যবহার

LIMIT 20 OFFSET 40 মানে প্রথম ৪০টা রো বাদ দিয়ে পরের ২০টা রিটার্ন করো (৩য় পেজ, ২০-প্রতি-পেজ ধরলে)। MySQL-এ শর্টহ্যান্ড LIMIT 40, 20 লেখা যায় কিন্তু এটা বিভ্রান্তিকর (প্রথম নাম্বারটা OFFSET, দ্বিতীয়টা COUNT — উল্টো মনে হয়) — তাই LIMIT ... OFFSET ... স্পষ্টভাবে লেখাই ভালো অভ্যাস।

DISTINCT — ডুপ্লিকেট বাদ দেওয়া

কোড	ব্যাখ্যা
SELECT DISTINCT primary_category_id FROM articles	যতগুলো ভিন্ন ক্যাটাগরিতে অন্তত একটা আর্টিকেল আছে তার তালিকা — ডুপ্লিকেট বাদ যায়
SELECT COUNT(DISTINCT user_id) FROM comments	কতজন ভিন্ন ইউজার কমেন্ট করেছে (মোট কমেন্ট সংখ্যা না, ইউনিক কমেন্টকারী সংখ্যা)

DISTINCT পুরো SELECT করা row-সেটের উপর কাজ করে, একটা মাত্র কলামের উপর না — SELECT DISTINCT a, b মানে (a,b) জোড়া হিসেবে ইউনিক, শুধু a বা শুধু b আলাদাভাবে না। এই ভুল বোঝাবুঝি প্রায়ই বাগের কারণ হয়।

- প্রশ্ন: `SELECT DISTINCT user_id, DATE(read_at) FROM read_histories` — এটা কী রিটার্ন করবে?

JOIN — সব ধরনের (সম্প্রসারিত)

INNER, LEFT, RIGHT, FULL OUTER — কনসেপ্ট গভীরভাবে

চিত্রের মাধ্যমে বোঝা — Venn Diagram অ্যানালজি

JOIN টাইপ	ফলাফলে কী থাকে
INNER JOIN	শুধু দুই টেবিলে মিল থাকা অংশ (Venn diagram-এর ওভারল্যাপ)
LEFT JOIN	বাম টেবিলের সবকিছু + মিল থাকলে ডান টেবিলের ডেটা, না থাকলে NULL
RIGHT JOIN	LEFT JOIN-এর উল্টো — ডান টেবিলের সবকিছু + মিল থাকলে বাম টেবিলের ডেটা
FULL OUTER JOIN	দুই টেবিলের সবকিছু, মিল না থাকলে যেদিকে নেই সেদিকে NULL (MySQL-এ সরাসরি নেই, LEFT+UNION+RIGHT দিয়ে বানাতে হয়)

বাস্তবে RIGHT JOIN খুবই কম ব্যবহার হয় — কারণ RIGHT JOIN A,B সবসময় LEFT JOIN B,A লিখে (টেবিলের ক্রম উল্টে) একই ফলাফল পাওয়া যায়, এবং সবাই LEFT JOIN পড়তে বেশি স্বচ্ছন্দ। তাই কনভেনশন হিসেবে এই প্রজেক্টে সবসময় LEFT JOIN ব্যবহার করা হয়েছে, RIGHT JOIN কোথাও নেই।

একাধিক JOIN চেইন করা

```
SELECT a.headline, u.first_name, c.name AS category
FROM articles a
JOIN users u ON a.created_by = u.id
JOIN categories c ON a.primary_category_id = c.id
LEFT JOIN article_translations at ON a.id = at.source_article_id
WHERE a.status = 'published'
```

তিন-চার-পাঁচটা JOIN চেইন করা স্বাভাবিক — প্রতিটা JOIN একটা নতুন টেবিল যোগ করে। লক্ষণীয়: প্রথম দুটো JOIN (creator, category আবশ্যিক) সাধারণ JOIN/INNER JOIN, শেষেরটা (translation,ঐচ্ছিক) LEFT JOIN — কারণ প্রতিটা আর্টিকেলের অনুবাদ নাও থাকতে পারে।

JOIN vs Subquery — কখন কোনটা পছন্দ করবে

পরিস্থিতি	পছন্দ
একাধিক টেবিলের কলাম একসাথে SELECT করতে হবে	JOIN
শুধু existence/count চেক করতে হবে, আসল ডেটা দরকার নেই	Subquery (EXISTS/IN)
একই সোর্স থেকে aggregate ও detail দুটোই লাগবে	JOIN + derived table/CTE

- প্রশ্ন: "প্রতিটা আর্টিকেলের হেডলাইন এবং তার মোট কमेंট সংখ্যা" — এটা LEFT JOIN + GROUP BY দিয়ে নাকি subquery দিয়ে করবে, কেন?

মডিউল ৫

Self-JOIN ও Hierarchical ডেটা

ট্রি-স্ট্রাকচার কুয়েরি করার তিনটা কৌশল

পদ্ধতি ১ — Self-JOIN (এক লেভেল)

```
SELECT child.name AS sub, parent.name AS main
FROM categories child
LEFT JOIN categories parent ON child.parent_id = parent.id
```

শুধু এক লেভেল উপরে/নিচে যেতে ভালো কাজ করে। দুই লেভেল (নাতি-নাতনি) দরকার হলে আরেকটা self-JOIN লাগবে, তিন লেভেলে আরেকটা — অজানা গভীরতার ট্রি-তে এই পদ্ধতি কাজ করে না।

পদ্ধতি ২ — Recursive CTE (MySQL 8+)

```
WITH RECURSIVE category_tree AS (
  SELECT id, name, parent_id, 0 AS depth
  FROM categories WHERE id = 1
  UNION ALL
  SELECT c.id, c.name, c.parent_id, ct.depth + 1
  FROM categories c
  JOIN category_tree ct ON c.parent_id = ct.id
)
SELECT * FROM category_tree
```

এটা যেকোনো গভীরতার ট্রি হাঁটতে পারে — প্রথম অংশ (anchor) শুরুর পয়েন্ট, দ্বিতীয় অংশ (recursive) বারবার নিজেকে রেফারেন্স করে গভীরে যায় যতক্ষণ না নতুন কোনো চাইল্ড না পাওয়া যায়।

সমস্যা: Recursive CTE প্রতিটা লেভেলে আলাদা আলাদা কুয়েরি চালায় (ভেতরে ভেতরে loop-এর মতো), তাই গভীর/বড় ট্রি-তে ধীরগতির হতে পারে, আর MySQL ঠিকমতো এটার জন্য ইনডেক্স অপটিমাইজেশন করতে পারে না যেমনটা সাধারণ কুয়েরিতে করে।

পদ্ধতি ৩ — Materialized Path (এই প্রজেক্টের পছন্দ)

```
SELECT * FROM categories
WHERE materialized_path LIKE '/1/5/%'
```

categories.materialized_path কলামে পুরো পূর্বপুরুষ চেইন স্ট্রিং হিসেবে সংরক্ষিত থাকে (যেমন /1/5/12/)। একটা মাত্র LIKE কুয়েরি দিয়েই যেকোনো গভীরতার পুরো সাব-ট্রি পাওয়া যায়, আর LIKE 'prefix%' প্যাটার্ন (শুরুতে % নেই) B-Tree ইনডেক্স ব্যবহার করতে পারে — তাই এটা দ্রুততম।

পদ্ধতি	পড়া (Read)	লেখা (Write/Move)	কখন ব্যবহার
Self-JOIN	ধীর, গভীরতা সীমিত	সহজ	নির্দিষ্ট এক লেভেল সম্পর্ক (parent-child শুধু)
Recursive CTE	মাঝারি, MySQL 8+ লাগে	সহজ	ছোট ট্রি, নমনীয়তা দরকার
Materialized Path	সবচেয়ে দ্রুত	জটিল (নোড move করলে পুরো সাব-ট্রির path আপডেট করতে হয়)	বড়, প্রায়ই-পড়া-হয়-কম-লেখা-হয় এমন ট্রি (categories, comments)

- প্রশ্ন: Materialized Path-এ একটা ক্যাটাগরিকে move (parent বদল) করলে কেন শুধু ঐ একটা রো না, তার নিচের পুরো সাব-ট্রির path আপডেট করতে হয়?

Aggregation — COUNT, SUM, AVG, GROUP BY (সম্প্রসারিত)

একাধিক গ্রুপিং কলাম ও সাধারণ ভুল

COUNT(*) vs COUNT(column) vs COUNT(DISTINCT column) — তিনটার পার্থক্য

ফর্ম	গণনা করে
COUNT(*)	মোট রো সংখ্যা, NULL সহ সবকিছু
COUNT(column)	শুধু সেই কলামে non-NULL মান আছে এমন রো
COUNT(DISTINCT column)	শুধু সেই কলামের ভিন্ন ভিন্ন non-NULL মান

```
SELECT
  COUNT(*) AS total_reactions,
  COUNT(user_id) AS logged_in_reactions,
  COUNT(DISTINCT user_id) AS unique_users
FROM article_reactions
WHERE article_id = ?
```

article_reactions.user_id nullable (অ্যানোনিমাস রিয়াকশনের জন্য) — তাই COUNT(*) সব রিয়াকশন গণনা করবে, COUNT(user_id) শুধু লগইন-করা-ইউজারের রিয়াকশন গণনা করবে (অ্যানোনিমাস বাদ), আর COUNT(DISTINCT user_id) বলবে কতজন ভিন্ন লগইন-ইউজার রিয়াক্ট করেছে।

একাধিক কলাম দিয়ে GROUP BY

```
SELECT primary_category_id, locale, COUNT(*) AS total
FROM articles
WHERE status = 'published'
GROUP BY primary_category_id, locale
```

এখন প্রতিটা (category, locale) জোড়ার জন্য আলাদা একটা গ্রুপ তৈরি হবে — যেমন "খেলাধুলা+বাংলা" একটা গ্রুপ, "খেলাধুলা+ইংরেজি" আরেকটা গ্রুপ।

সাধারণ ভুল — Non-aggregated কলাম SELECT করা

MySQL ডিফল্টভাবে (ONLY_FULL_GROUP_BY মোড চালু থাকলে, যেটা MySQL 5.7+ থেকে ডিফল্ট) এই ভুল ধরে ফেলে এরর দিয়ে: `SELECT headline, COUNT(*) FROM articles GROUP BY primary_category_id` — এখানে headline না GROUP BY-তে আছে, না aggregate function-এর ভেতরে, তাই MySQL জানে না কোন headline দেখাবে (একটা গ্রুপে একাধিক headline থাকতে পারে)। সমাধান: হয় headline-কে GROUP BY-তে যোগ করো, নয়তো MIN(headline)/MAX(headline) দিয়ে র‍্যাপ করো।

- প্রশ্ন: "প্রতিটা ক্যাটাগরিতে সবচেয়ে সাম্প্রতিক আর্টিকেলের হেডলাইন ও তার প্রকাশের তারিখ" — এই কুয়েরিটা GROUP BY দিয়ে ঠিকমতো লেখা যায় না কেন (হিন্ট: MAX(published_at) দিয়ে তারিখ পাওয়া যাবে, কিন্তু headline?), আর সঠিক সমাধান কী হবে (Window Function — অধ্যায় ১০ দেখো)?

HAVING — গ্রুপ ফিল্টারিং গভীরভাবে

WHERE বনাম HAVING-এর প্রতিটা সূক্ষ্ম পার্থক্য

একসাথে WHERE ও HAVING

```
SELECT user_id, COUNT(DISTINCT article_id) AS reads
FROM read_histories
WHERE read_at >= '2026-08-01'
GROUP BY user_id
HAVING COUNT(DISTINCT article_id) >= 5
```

ক্লজ	কাজ করে কখন	এই কুয়েরিতে
WHERE	গ্রুপ তৈরি হওয়ার আগে, প্রতিটা raw রো-এর উপর	শুধু আগস্ট-২০২৬-এর পরের read_histories রো বাছাই করছে
GROUP BY	বাছাই করা রোগুলোকে user_id অনুযায়ী গ্রুপ করছে	—
HAVING	প্রতিটা গ্রুপের উপর (গ্রুপ তৈরি হওয়ার পরে)	শুধু যেসব ইউজারের গ্রুপে ৫+ ইউনিক আর্টিকেল আছে তাদের রাখছে

HAVING-এ কি non-aggregate কন্ডিশন লেখা যায়?

```
SELECT primary_category_id, COUNT(*) AS total
FROM articles
GROUP BY primary_category_id
HAVING primary_category_id > 5 AND COUNT(*) > 10
```

হ্যাঁ, টেকনিক্যালি HAVING-এ GROUP BY কলামও ব্যবহার করা যায় (primary_category_id > 5) — কিন্তু এটা অপ্টিমাইজেশনের দিক থেকে ভুল অভ্যাস, কারণ MySQL তখন সব রো গ্রুপ করার পরে ফিল্টার করে, যেখানে primary_category_id > 5 আসলে WHERE-এ থাকলে গ্রুপ করার আগেই অপ্টিমাইজেশন রো বাদ দিয়ে দিত (কম কাজ)। নিয়ম: যা WHERE দিয়ে করা সম্ভব তা সবসময় WHERE-এ লেখো, শুধু aggregate function-নির্ভর কন্ডিশনের জন্য HAVING রাখো।

```
-- ভালো (দ্রুত): প্রথমে ফিল্টার, তারপর গ্রুপ
SELECT primary_category_id, COUNT(*) AS total
FROM articles
WHERE primary_category_id > 5
GROUP BY primary_category_id
HAVING COUNT(*) > 10
```

- প্রশ্ন: "যেসব ফ্রিল্যান্সারের মোট ইনভয়েন্স এমআউন্ট ৫০,০০০-এর বেশি এবং যাদের চুক্তি এখনো সক্রিয়" — কোন অংশ WHERE-এ যাবে, কোন অংশ HAVING-এ?

মডিউল ৮

Subquery — Scalar, IN, EXISTS, Correlated

কুয়েরির ভেতরে কুয়েরি — চার ধরনের ব্যবহার

১. Scalar Subquery — একটা মাত্র মান রিটার্ন করে

```
SELECT headline, view_count,
       (SELECT AVG(view_count) FROM articles WHERE status='published') AS
       avg_views
FROM articles
WHERE status = 'published'
```

প্রতিটা সারির পাশে গড় ভিউ-কাউন্ট দেখাচ্ছে (একটা constant মান, যা প্রতিটা রো-তে একই)। এখানে subquery ঠিক একটা মান রিটার্ন করে — একাধিক রো/কলাম রিটার্ন করলে MySQL এরর দেবে।

২. Non-correlated Subquery (IN) — একবার execute হয়

```
SELECT * FROM articles
WHERE primary_category_id IN (
  SELECT id FROM categories WHERE is_sensitive = 1
)
```

ভেতরের কুয়েরিটা বাইরের কুয়েরি থেকে সম্পূর্ণ স্বাধীন — এটা একবার execute হয়ে একটা তালিকা বানায় (is_sensitive ক্যাটাগরির ID-গুলো), তারপর বাইরের কুয়েরি সেই স্থির তালিকার সাথে প্রতিটা রো মেলায়।

৩. Correlated Subquery — প্রতিটা বাইরের রো-এর জন্য আলাদা execute হয়

```
SELECT a.headline, a.view_count
FROM articles a
WHERE a.view_count > (
  SELECT AVG(a2.view_count)
  FROM articles a2
  WHERE a2.primary_category_id = a.primary_category_id
)
```

লক্ষ করো — ভেতরের subquery-তে a.primary_category_id ব্যবহার হয়েছে, যেটা বাইরের কুয়েরির টেবিল থেকে আসছে। এই কারণে এটা "correlated" — প্রতিটা বাইরের রো-এর জন্য ভেতরের কুয়েরি নতুন করে চলে (নিজ ক্যাটাগরির গড় বের করতে)। এটা conceptually একটা for-loop-এর মতো, তাই বড় টেবিলে এটা ধীরগতির হতে পারে — সমতুল্য কাজ প্রায়ই Window Function দিয়ে (অধ্যায় ১০) অনেক দ্রুত করা যায়।

৪. EXISTS subquery — শুধু সত্য/মিথ্যা চেক

```
SELECT u.email FROM users u
WHERE EXISTS (
  SELECT 1 FROM subscriptions s
  WHERE s.user_id = u.id AND s.status = 'active'
)
```

যেসব ইউজারের অন্তত একটা সক্রিয় সাবস্ক্রিপশন আছে — subquery-তে SELECT 1 লেখা হয়েছে কারণ আসল ভ্যালু কিছু আসে না, শুধু "একটা রো পাওয়া গেছে কিনা" সেটাই গুরুত্বপূর্ণ (কনভেনশন হিসেবে SELECT 1 লেখা হয়, SELECT * না — সামান্য দ্রুত কারণ MySQL কোনো কলাম resolve করতে হয় না)।

- প্রশ্ন: correlated subquery আর JOIN দিয়ে একই কাজ করলে কোনটা সাধারণত দ্রুত, আর কেন MySQL optimizer অনেক correlated subquery-কে ভেতরে ভেতরে JOIN-এ রূপান্তর করার চেষ্টা করে?

CTE (Common Table Expression)

জটিল কুয়েরি ধাপে ধাপে ভাগ করা

একাধিক CTE একসাথে (chaining)

```
WITH
published_articles AS (
  SELECT * FROM articles WHERE status = 'published'
),
category_stats AS (
  SELECT primary_category_id, COUNT(*) AS total
  FROM published_articles
  GROUP BY primary_category_id
)
SELECT c.name, cs.total
FROM category_stats cs
JOIN categories c ON cs.primary_category_id = c.id
ORDER BY cs.total DESC
```

একটা CTE আরেকটা CTE-কে রেফারেন্স করতে পারে (এখানে `category_stats`, `published_articles` ব্যবহার করছে) — জটিল লজিককে ছোট ছোট নামযুক্ত ধাপে ভাগ করে `readable` রাখার সবচেয়ে ভালো উপায়।

CTE vs Subquery — কেন CTE readable

বিষয়	Subquery (nested)	CTE
Readability	গভীরভাবে nested হলে পড়া কঠিন হয়ে যায়	উপর থেকে নিচে ধাপে ধাপে পড়া যায়, নাম দিয়ে বোঝা যায় প্রতিটা অংশ কী করছে
পুনর্ব্যবহার	একই subquery একাধিকবার লিখতে হয়	একবার সংজ্ঞায়িত করে একাধিকবার রেফারেন্স করা যায়
Performance	একই	MySQL 8-এ প্রায় একই (optimizer উভয়কে একইভাবে ট্রিট করে বেশিরভাগ ক্ষেত্রে)

Recursive CTE — পুনরাবৃত্তি

```
WITH RECURSIVE thread AS (
  SELECT id, parent_id, body, 0 AS depth
  FROM comments WHERE id = 100
  UNION ALL
  SELECT c.id, c.parent_id, c.body, t.depth + 1
  FROM comments c
  JOIN thread t ON c.parent_id = t.id
)
SELECT * FROM thread ORDER BY depth
```

কমেন্ট #১০০-এর নিচে থাকা পুরো রিপ্লাই-চেইন (যেকোনো গভীরতা পর্যন্ত) বের করার একটা পদ্ধতি — যদিও এই প্রজেক্টে `comments` টেবিলেও `materialized_path` আছে যা এই একই কাজ দ্রুত করতে পারে (অধ্যায় ৫ দেখো)।

- প্রশ্ন: recursive CTE-তে `UNION ALL` কেন ব্যবহার হয়, `UNION` না (হিট: *performance ও duplicate handling*)?

মডিউল ১০

Window Functions (সম্প্রসারিত)

RANK, NTILE, FIRST_VALUE ও আরও গভীরে

OVER() কুজের তিনটা অংশ

অংশ	কাজ
PARTITION BY	GROUP BY-এর মতো, কিন্তু রো একত্রিত হয় না — প্রতিটা রো আলাদা থাকে
ORDER BY	পার্টিশনের ভেতরে রো-গুলোর ক্রম ঠিক করে (RANK, LAG/LEAD-এর জন্য জরুরি)
ROWS/RANGE BETWEEN	কোন রো-গুলো নিয়ে গণনা হবে তার সীমা (যেমন "আগের ৩টা রো")

RANK() vs DENSE_RANK() vs ROW_NUMBER() — টাই হলে পার্থক্য

```
SELECT headline, view_count,
       RANK() OVER (ORDER BY view_count DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY view_count DESC) AS dense_rnk,
       ROW_NUMBER() OVER (ORDER BY view_count DESC) AS row_num
FROM articles
```

view_count	RANK()	DENSE_RANK()	ROW_NUMBER()
১০০০	১	১	১
৯০০ (টাই)	২	২	২
৯০০ (টাই)	২	২	৩
৮০০	৪	৩	৪

RANK() টাইয়ের পর একটা "গ্যাপ" রাখে (২, ২, ৪ — ৩ বাদ পড়ে গেছে), DENSE_RANK() গ্যাপ রাখে না (২, ২, ৩), আর ROW_NUMBER() টাই থাকলেও প্রতিটিকে আলাদা নাম্বার দেয় (২, ৩) — টাই ভাঙার জন্য ORDER BY-তে একাধিক কলাম (secondary sort) না দিলে ROW_NUMBER()-এর ক্রম অনির্দিষ্ট (arbitrary) হতে পারে।

FIRST_VALUE() / LAST_VALUE() — পার্টিশনের প্রথম/শেষ মান

```
SELECT article_id, snapshot_date, total_views,
       FIRST_VALUE(total_views) OVER (
         PARTITION BY article_id ORDER BY snapshot_date
       ) AS first_day_views
FROM content_performance_snapshots
```

প্রতিটা আর্টিকেলের প্রথম দিনের ভিউয়ের সাথে পরের প্রতিটা দিনের ভিউ তুলনা করার জন্য উপকারী।

NTILE() — সমান ভাগে বাঁটা (পার্সেন্টাইল/কোয়ার্টাইল)

```
SELECT user_id, total_reads,
       NTILE(4) OVER (ORDER BY total_reads DESC) AS quartile
FROM (
  SELECT user_id, COUNT(*) AS total_reads
  FROM read_histories GROUP BY user_id
) t
```

সব ইউজারকে তাদের রিডিং অ্যাক্টিভিটি অনুযায়ী ৪টা সমান দলে ভাগ করে (**quartile=1** সবচেয়ে বেশি পড়ে, **quartile=4** সবচেয়ে কম) — মার্কেটিং সেগমেন্টেশনে (যেমন কাদের রিটেনশন অফার দেখাবে) উপকারী।

Frame Clause — ROWS BETWEEN দিয়ে চলমান গড়/যোগফল

```
SELECT snapshot_date, total_views,  
       AVG(total_views) OVER (  
         ORDER BY snapshot_date  
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW  
       ) AS rolling_7day_avg  
FROM content_performance_snapshots  
WHERE article_id = ?
```

৭-দিনের চলমান গড় (rolling average) — প্রতিটা দিনের জন্য সেই দিন সহ আগের ৬ দিনের গড় বের করে। এটা "smoothing" করে — একদিনের হঠাৎ স্পাইক/ড্রপ পুরো ট্রেন্ডকে ভুলভাবে প্রভাবিত করে না।

- প্রশ্ন: **NTILE(4)** আর কোয়ার্টাইলের ধারণা ব্যবহার করে কীভাবে "টপ ২৫% সবচেয়ে সক্রিয় পাঠক" বের করবে?

Set Operations — UNION, INTERSECT, EXCEPT

একাধিক কুয়েরির ফলাফল একত্রিত করা

UNION vs UNION ALL

<pre>SELECT email, 'newsletter' AS source FROM subscribers UNION SELECT email, 'user' AS source FROM users WHERE email IS NOT NULL</pre>	
অপারেটর	আচরণ
UNION	দুই কুয়েরির ফলাফল একত্রিত করে, ডুপ্লিকেট বাদ দেয় (ভেতরে ভেতরে একটা DISTINCT চালায় — বাড়তি খরচ)
UNION ALL	দুই কুয়েরির ফলাফল একত্রিত করে, ডুপ্লিকেট রাখে — দ্রুত, কারণ deduplication-এর কাজ করতে হয় না

নিয়ম: যদি নিশ্চিতভাবে জানো ডুপ্লিকেট থাকবে না, অথবা ডুপ্লিকেট থাকলেও সমস্যা নেই, তাহলে সবসময় **UNION ALL** ব্যবহার করো — **UNION**-এর deduplication পদক্ষেপ বড় ডেটাসেটে খরচসাপেক্ষ। এই প্রজেক্টে Recursive CTE-তেও (অধ্যায় ৯) তাই **UNION ALL** ব্যবহার হয়েছে, **UNION** না।

নিয়ম — কলাম সংখ্যা ও টাইপ মিলতে হবে

UNION করা সব **SELECT**-এ কলামের সংখ্যা এবং সামঞ্জস্যপূর্ণ (compatible) ডেটা টাইপ থাকতে হবে — প্রথম **SELECT**-এর কলাম নাম ফলাফলে ব্যবহৃত হয়, বাকিগুলোর নাম উপেক্ষিত হয়।

MySQL-এ INTERSECT ও EXCEPT (8.0.31+)

অপারেটর	অর্থ	উদাহরণ
INTERSECT	দুই কুয়েরির কমন ফলাফল (দুটোতেই আছে এমন)	যেসব ইউজার এই সপ্তাহে পড়েছে INTERSECT যারা কमेंটও করেছে
EXCEPT	প্রথম কুয়েরিতে আছে কিন্তু দ্বিতীয়টাতে নেই	যেসব সাবস্ক্রাইবার আছে EXCEPT যারা এই মাসে লগইন করেছে (নিষ্ক্রিয় সাবস্ক্রাইবার খুঁজতে)

পুরনো MySQL ভার্সনে (8.0.31-এর আগে) **INTERSECT/EXCEPT** ছিল না — সেক্ষেত্রে **INNER JOIN** দিয়ে **INTERSECT** আর **LEFT JOIN + WHERE ... IS NULL** দিয়ে **EXCEPT** বাস্তবায়ন করতে হতো। এই প্রজেক্টের স্ট্যাক (MySQL 8) এই অপারেটরগুলো সরাসরি সাপোর্ট করে।

- প্রশ্ন: "যেসব ইউজার সাবস্ক্রাইবার কিন্তু গত ৩০ দিনে একটাও আর্টিকেল পড়েনি" — **EXCEPT** দিয়ে কীভাবে লিখবে?

CASE WHEN ও Conditional Logic

SQL-এর ভেতরে *if-else*

Simple CASE ও Searched CASE

```
SELECT headline,
CASE
  WHEN view_count > 100000 THEN 'viral'
  WHEN view_count > 10000 THEN 'popular'
  WHEN view_count > 1000 THEN 'normal'
  ELSE 'low'
END AS popularity_tier
FROM articles
```

উপর থেকে নিচে প্রতিটা WHEN শর্ত চেক হয়, প্রথম যেটা সত্য হয় সেটার THEN-এর মান রিটার্ন হয় — ELSE না থাকলে কোনো শর্ত না মিললে NULL রিটার্ন হয়।

SELECT-এর ভেতরে Conditional Aggregation (pivot প্যাটার্ন)

```
SELECT primary_category_id,
SUM(CASE WHEN locale='bn' THEN 1 ELSE 0 END) AS bangla_count,
SUM(CASE WHEN locale='en' THEN 1 ELSE 0 END) AS english_count
FROM articles
GROUP BY primary_category_id
```

একটা খুবই শক্তিশালী প্যাটার্ন — একটা কুয়েরিতেই একাধিক শর্তের গণনা "পাশাপাশি কলামে" (pivot টেবিলের মতো) দেখানো যায়, একাধিকবার একই টেবিল কুয়েরি না করে।

এই প্যাটার্ন এই প্রজেক্টে ব্যবহার হতে পারে: *ad_analytics*-এ *device*-অনুযায়ী *impression* ব্রেকডাউন, অথবা *users*-এ *acquisition_source*-অনুযায়ী কনভার্সন রেট — এক কুয়েরিতেই ড্যাশবোর্ডের একাধিক কলাম তৈরি করা যায়।

WHERE-এর ভেতরে CASE WHEN (কম ব্যবহৃত কিন্তু কাজে লাগে)

```
SELECT * FROM subscriptions
WHERE CASE
  WHEN status = 'trialing' THEN trial_ends_at < NOW()
  WHEN status = 'active' THEN ends_at < NOW()
  ELSE FALSE
END
```

বিভিন্ন status-এর জন্য "মেয়াদ শেষ" মানে ভিন্ন কলাম চেক করা — trial অবস্থায় trial_ends_at, active অবস্থায় ends_at।

- প্রশ্ন: CASE WHEN দিয়ে "রিডিং স্পিড অনুযায়ী ইউজারদের slow/medium/fast — তিন ভাগে ভাগ করে" এই কুয়েরি কীভাবে লিখবে (user_preferences.reading_speed_wpm ব্যবহার করে)?

String Functions

টেক্সট ম্যানিপুলেশনের গুরুত্বপূর্ণ ফাংশন

সবচেয়ে বেশি ব্যবহৃত String Function

ফাংশন	কাজ	উদাহরণ
CONCAT()	জোড়া লাগানো	CONCAT(first_name, ' ', last_name) AS full_name
LENGTH() / CHAR_LENGTH()	বাইট দৈর্ঘ্য / ক্যারেটার দৈর্ঘ্য (multi-byte UTF-8-তে ভিন্ন)	CHAR_LENGTH(headline)
SUBSTRING()	অংশ কেটে নেওয়া	SUBSTRING(content_preview, 1, 100)
UPPER() / LOWER()	বড়/ছোট হাতের অক্ষর	LOWER(email) — কেস-ইনসেনসিটিভ তুলনার জন্য
TRIM()	শুরু/শেষের স্পেস বাদ দেওয়া	TRIM(search_term)
REPLACE()	অংশ প্রতিস্থাপন	REPLACE(slug, '_', '-')
CONCAT_WS()	নির্দিষ্ট সেপারেটর দিয়ে জোড়া (NULL এড়িয়ে চলে)	CONCAT_WS(' ', city, country)

CHAR_LENGTH() বনাম LENGTH()-এর পার্থক্য গুরুত্বপূর্ণ বাংলা টেক্সটের জন্য — বাংলা ক্যারেটার utf8mb4-এ ৩-৪ বাইট নেয়, তাই LENGTH() (বাইট গণনা করে) একটা বাংলা হেডলাইনের জন্য CHAR_LENGTH()-এর (আসল অক্ষর গণনা করে) চেয়ে অনেক বড় সংখ্যা দেখাবে। "headline ৫০ ক্যারেটারের কম কিনা" চেক করতে সবসময় CHAR_LENGTH() ব্যবহার করা উচিত।

LIKE প্যাটার্নের বিস্তারিত

প্যাটার্ন	মানে
'নির্বাচন%'	'নির্বাচন' দিয়ে শুরু (ইনডেক্স ব্যবহার করতে পারে)
'%নির্বাচন'	'নির্বাচন' দিয়ে শেষ (ইনডেক্স ব্যবহার করতে পারে না)
'%নির্বাচন%'	যেকোনো জায়গায় 'নির্বাচন' আছে (ইনডেক্স ব্যবহার করতে পারে না)
'নির্_'	'নির্' দিয়ে শুরু, তারপর ঠিক ৪টা যেকোনো ক্যারেটার (_ একটা ক্যারেটার বোঝায়)

REGEXP — প্যাটার্ন ম্যাচিং আরও শক্তিশালী

```
SELECT * FROM users
WHERE phone REGEXP '^\\+8801[0-9]{9}$'
```

বাংলাদেশি ফোন নাম্বার ফরম্যাট ভ্যালিডেট করার উদাহরণ — REGEXP LIKE-এর চেয়ে অনেক বেশি জটিল প্যাটার্ন সাপোর্ট করে, কিন্তু LIKE-এর চেয়ে ধীরগতির এবং ইনডেক্স ব্যবহার করতে পারে না, তাই বড় টেবিলে সতর্কতার সাথে ব্যবহার করা উচিত।

- প্রশ্ন: slug কলামে ইনডেক্স আছে ধরে নিলে, WHERE slug LIKE 'bangladesh%' আর WHERE slug LIKE '%bangladesh' — কোনটা ইনডেক্স ব্যবহার করবে, কেন?

Date/Time Functions

সময়-ভিত্তিক প্রায় প্রতিটা রিপোর্টের ভিত্তি

বর্তমান সময় ও অংশ বের করা

ফাংশন	কাজ
NOW() / CURDATE()	বর্তমান তারিখ+সময় / শুধু তারিখ
YEAR() / MONTH() / DAY()	তারিখ থেকে অংশ বের করা
DATE()	DATETIME থেকে শুধু তারিখের অংশ (সময় বাদ)
DAYNAME() / DAYOFWEEK()	বারের নাম / সপ্তাহের কততম দিন

তারিখ গণনা — DATE_ADD, DATE_SUB, DATEDIFF

কোড	ব্যাখ্যা
SELECT * FROM classified_ads WHERE expires_at < NOW()	মেয়াদ শেষ হওয়া বিজ্ঞাপন খোঁজা
SELECT * FROM otps WHERE created_at < NOW() - INTERVAL 10 MINUTE	১০ মিনিটের বেশি পুরনো OTP (এক্সপায়ার্ড হিসেবে ধরা)
SELECT DATEDIFF(NOW(), published_at) AS days_since_publish FROM articles	প্রকাশের পর কত দিন হয়েছে

এই প্রজেক্টে সবচেয়ে গুরুত্বপূর্ণ প্যাটার্ন — "এই মাসের শুরু"

```
WHERE read_at >= DATE_FORMAT(NOW(), '%Y-%m-01')
```

DATE_FORMAT(NOW(), '%Y-%m-01') বর্তমান মাসের ১ তারিখ বের করে (যেমন 2026-08-01) — *paywall metering-এ (মডিউল-৮)* "এই মাসে কয়টা আর্টিকেল পড়েছে" গণনা করতে বারবার এই প্যাটার্ন লাগে। বিকল্প: *DATE_SUB(NOW(), INTERVAL DAY(NOW())-1 DAY)* — একই ফলাফল, ভিন্ন লেখার ধরন।

TIMESTAMPDIFF — নির্দিষ্ট এককে সময়ের পার্থক্য

```
SELECT TIMESTAMPDIFF(SECOND, created_at, NOW()) AS age_seconds
FROM personal_access_tokens
WHERE TIMESTAMPDIFF(DAY, created_at, NOW()) > 90
```

personal_access_tokens-এ "৯০ দিনের বেশি পুরনো টোকেন রোট্টে করো" স্কেজিউলড জবের ভিত্তি এটাই (মডিউল-১ দেখো)।

DATE_FORMAT — উপস্থাপনার জন্য ফরম্যাট করা

```
SELECT DATE_FORMAT(published_at, '%d %M %Y, %h:%i %p') AS display_date
FROM articles
```

ডেটাবেজে সবসময় স্ট্যান্ডার্ড ফরম্যাটে (TIMESTAMP/DATETIME) সংরক্ষণ করা হয়, ইউজারকে দেখানোর সময় DATE_FORMAT (বা অ্যাপ্লিকেশন লেয়ারে Carbon/moment.js দিয়ে সুন্দর ফরম্যাটে রূপান্তর করা হয় — কখনো ডেটাবেজে পি-ফরম্যাটেড স্ট্রিং হিসেবে তারিখ সংরক্ষণ করা উচিত না, কারণ তখন তারিখ-ভিত্তিক তুলনা/গণনা করা কঠিন হয়ে যায়।

- প্রশ্ন: "পত ৭ দিনে প্রতিদিন কতগুলো আর্টিকেল প্রকাশিত হয়েছে" — `DATE()` ও `GROUP BY` দিয়ে এই রিপোর্ট কীভাবে লিখবে?

NULL হান্ডলিং

SQL-এর সবচেয়ে বিভ্রান্তিকর কনসেপ্ট

NULL মানে "অজানা" — **false** না, খালি স্ট্রিং না

NULL কোনো মান না, এটা "মান নেই" বা "অজানা" এর প্রতিনিধিত্ব করে। এই কারণে NULL-এর সাথে যেকোনো তুলনা ($=$, $\neq$, $<$, $>$) ফলাফল দেয় NULL (true বা false না, "অজানা")। যেহেতু WHERE ক্লজ শুধু true হলে রো রাখে, NULL ফলাফলযুক্ত শর্ত সবসময় রো বাদ দিয়ে দেয়।

এক্সপ্রেশন	ফলাফল
NULL = NULL	NULL (true না!)
NULL != NULL	NULL (false না!)
NULL + 5	NULL
CONCAT('হ্যালো ', NULL)	NULL (পুরো এক্সপ্রেশন NULL হয়ে যায়)
NULL IS NULL	TRUE (এটাই একমাত্র সঠিক উপায়)

COALESCE() — প্রথম **non-NULL** মান

```
SELECT headline,
       COALESCE(seo_title, headline) AS display_title,
       COALESCE(byline_override, u.first_name) AS byline
FROM articles a
JOIN article_author aa ON a.id = aa.article_id
JOIN users u ON aa.user_id = u.id
```

COALESCE() একাধিক আর্গুমেন্ট নেয়, বাম থেকে প্রথম যেটা NULL না সেটা রিটার্ন করে — "যদি এটা না থাকে তাহলে ওটা ব্যবহার করো" প্যাটার্নের জন্য আদর্শ। এই প্রজেক্টে seo_title (nullable override) না থাকলে headline দেখানো, byline_override না থাকলে ইউজারের আসল নাম দেখানো — এরকম প্রায় সব "override" কলামে এই প্যাটার্ন লাগে।

IFNULL() vs COALESCE()

IFNULL(a, b) ঠিক দুটো আর্গুমেন্ট নেয় (MySQL-স্পেসিফিক ফাংশন), COALESCE(a, b, c, ...) যেকোনো সংখ্যক আর্গুমেন্ট নেয় এবং ANSI SQL স্ট্যান্ডার্ড (অন্য ডেটাবেজেও কাজ করে — PostgreSQL, SQL Server)। দুই বা তার বেশি অপশনের ফলবাক্য চেইন দরকার হলে COALESCE ব্যবহার করাই ভালো অভ্যাস, পোর্টেবিলিটির জন্যও।

NULL-safe তুলনা — $<=>$

```
SELECT * FROM users
WHERE email <=> NULL
```

সাধারণ = অপারেটর NULL-এর সাথে কখনো true দেয় না, কিন্তু $<=>$ (NULL-safe equal) দেয় — WHERE email $<=>$ NULL ঠিক WHERE email IS NULL-এর মতোই কাজ করে। এটা বিশেষভাবে উপকারী যখন দুটো কলাম তুলনা করতে হয় এবং উভয়েই NULL হতে পারে (a $<=>$ b সাধারণ a = b-এর চেয়ে নিরাপদ যদি NULL সম্ভাবনা থাকে)।

- প্রশ্ন: WHERE deleted_at != '2026-01-01' — এই কুয়েরি deleted_at=NULL এমন রো রিটার্ন করবে কি? কেন?

Index ও Query Planner

কুয়েরি দ্রুত করার আসল রহস্য

Index আসলে কী — B-Tree স্ট্রাকচার

একটা বইয়ের সূচিপত্র যেমন প্রতিটা পাতা না উল্টে সরাসরি বিষয়ে পৌঁছে দেয়, ডেটাবেজ ইনডেক্সও তেমন — একটা আলাদা, সাজানো ডেটা স্ট্রাকচার (সাধারণত B-Tree) যা কলামের ভ্যালু থেকে সরাসরি রো-এর অবস্থানে নিয়ে যায়। ইনডেক্স ছাড়া MySQL-কে "full table scan" করতে হয় — প্রতিটা রো একে একে চেক করা, যা বড় টেবিলে ধ্বংসাত্মকভাবে ধীর।

B-Tree ইনডেক্স সাজানো (sorted) থাকে — এই কারণে ইনডেক্স শুধু = তুলনায় না, রেঞ্জ কুয়েরিতেও (>, <, BETWEEN) এবং ORDER BY-তেও সাহায্য করে। কিন্তু এই একই কারণে LIKE '%text' (শুরুতে wildcard) ইনডেক্স ব্যবহার করতে পারে না — কারণ "শেষে কী আছে" দিয়ে সাজানো তালিকায় দ্রুত খোঁজা সম্ভব না।

Composite Index — Leftmost Prefix Rule

ইনডেক্স	কাজ করবে	কাজ করবে না
INDEX(status, published_at)	WHERE status=? WHERE status=? AND published_at=? ORDER BY status, published_at	WHERE published_at>? (status ছাড়া একা) ORDER BY published_at (status ছাড়া একা)

leftmost কলাম (এখানে status) ছাড়া বাকি কলাম একা ব্যবহার করলে ইনডেক্স কাজ করে না — ঠিক যেমন ফোন-বইয়ে (নাম দিয়ে সাজানো) কারো ফোন নাম্বার দিয়ে সরাসরি নাম খোঁজা যায় না, প্রথম থেকে খুঁজতে হয়। এই কারণেই articles-এ idx_published_listing (status, published_at DESC) এই নির্দিষ্ট অর্ডারে বানানো — কারণ প্রতিটা লিস্টিং কুয়েরি সবসময় status দিয়ে ফিল্টার শুরু করে।

EXPLAIN পড়া শেখা

<pre>EXPLAIN SELECT * FROM articles WHERE status = 'published' ORDER BY published_at DESC LIMIT 20;</pre>	
কলাম	কী বোঝায়
type	অ্যাক্সেসের ধরন — const/eq_ref (সবচেয়ে ভালো) > ref > range > index > ALL (সবচেয়ে খারাপ, full scan)
key	কোন ইনডেক্স আসলে ব্যবহার হচ্ছে (NULL মানে কোনো ইনডেক্স ব্যবহার হয়নি)
rows	MySQL অনুমান করছে কতগুলো রো স্ক্যান করতে হবে — যত কম তত ভালো
Extra	"Using filesort" বা "Using temporary" দেখলে সতর্ক হও — এগুলো বাড়তি কাজ যা ইনডেক্স দিয়ে এড়ানো সম্ভব হতে পারে

Covering Index — সবচেয়ে দ্রুত সম্ভব কুয়েরি

যদি কুয়েরির প্রয়োজনীয় সব কলাম ইনডেক্সেই থাকে (SELECT-এর কলামসহ), MySQL মূল টেবিলে ফিরে না গিয়েই ইনডেক্স থেকে সরাসরি উত্তর দিতে পারে — EXPLAIN-এ Extra কলামে "Using index" দেখা যায়, এটাই সবচেয়ে দ্রুত সম্ভব read।

কখন ইনডেক্স বানাবে না

- খুব ছোট টেবিলে (কয়েকশ রো) — full scan-ও যথেষ্ট দ্রুত, ইনডেক্স মেইনটেইন করার খরচ লাভের চেয়ে বেশি।
- যে কলামে write অনেক বেশি কিন্তু সেই কলাম দিয়ে filter/sort কম হয় — প্রতিটা INSERT/UPDATE-এ ইনডেক্সও আপডেট করতে হয়, তাই বেশি ইনডেক্স মানে ধীরগতির write।

- খুব কম cardinality-র কলামে একা (যেমন is_active — শুধু 0/1) — ইনডেক্স খুব একটা সাহায্য করে না, কারণ অর্ধেক টেবিলই "true" হতে পারে।
- প্রশ্ন: EXPLAIN-এ type=ALL দেখলে এর মানে কী, আর এটা ঠিক করার প্রথম পদক্ষেপ কী হওয়া উচিত?

N+1 সমস্যা ও Eager Loading

সবচেয়ে সাধারণ পারফরম্যান্স ভুল

সমস্যাটা কী

২০টা আর্টিকেল দেখাতে হবে, প্রতিটার সাথে **author**-এর নাম। ভুল পদ্ধতি: প্রথমে ২০টা আর্টিকেল আনতে ১টা কুয়েরি, তারপর প্রতিটার জন্য আলাদা করে **author** খুঁজতে আরও ২০টা কুয়েরি — মোট ২১টা কুয়েরি (**N+1**, যেখানে **N=20**)।

ভুল (N+1)	সঠিক (Eager Loading)
প্রতিটা আর্টিকেলের লুপে: <code>SELECT * FROM users WHERE id=?</code> (২০ বার আলাদা কল)	<code>SELECT * FROM users WHERE id IN (৫,১২,৭,...)</code> (১ বার, ব্যাচে)

এই সমস্যা ডেভেলপমেন্টে ধরা পড়ে না (কয়েকটা টেস্ট রো দিয়ে পার্থক্য বোঝা যায় না), শুধু প্রোডাকশনে বড় ডেটাসেটে ধীরগতি প্রকাশ পায়। **Laravel**-এ `Article::with('author')->get()` করলে ভেতরে ভেতরে ঠিক এই দুই-কুয়েরি ব্যাচিং হয়।

গভীর Nested Relationship

```
-- Laravel: Article::with('author.institutionalAccount', 'category')->get();
-- ভেতরে চলে:
SELECT * FROM articles WHERE status='published' LIMIT 20;
SELECT * FROM users WHERE id IN (...);
SELECT * FROM institutional_accounts WHERE id IN (...);
SELECT * FROM categories WHERE id IN (...);
```

প্রতিটা লেভেলের রিলেশনের জন্য একটা করে ব্যাচ কুয়েরি — গভীরতা যতই হোক, মোট কুয়েরি সংখ্যা সবসময় "লেভেল সংখ্যা", "রো সংখ্যা" না। এটাই **N+1**-এর আসল সমাধান।

- প্রশ্ন: যদি ২০টা আর্টিকেলের প্রতিটার ৫টা করে ট্যাগ থাকে (`article_tag pivot` দিয়ে *many-to-many*), *eager loading* ছাড়া কতগুলো কুয়েরি লাগবে, আর *eager loading* দিয়ে কতগুলো?

Full-text Search ও JSON কুয়েরি

সাধারণ WHERE = দিয়ে যা সম্ভব না

LIKE কেন যথেষ্ট না

পদ্ধতি	সমস্যা/সুবিধা
WHERE headline LIKE '%নির্বাচন%'	কাজ করে কিন্তু ইনডেক্স ব্যবহার করতে পারে না (leading wildcard) — বড় টেবিলে ধীরগতির
WHERE MATCH(headline, sub_headline) AGAINST('নির্বাচন' IN NATURAL LANGUAGE MODE)	FULLTEXT ইনডেক্স ব্যবহার করে, relevance স্কোর দেয়, দ্রুত — শব্দ-ভিত্তিক (word-based), সাব-স্ট্রিং ম্যাচ না

এই প্রজেক্টে আসল সার্চ Meilisearch করে (typo-tolerant, বাংলা সাপোর্ট ভালো), কিন্তু articles টেবিলে FULLTEXT(headline, sub_headline) ইনডেক্সও fallback হিসেবে আছে।

Boolean Mode — সার্চ অপারেটর

```
SELECT * FROM articles
WHERE MATCH(headline, sub_headline)
      AGAINST(' +নির্বাচন -বিতর্কিত' IN BOOLEAN MODE)
```

+ মানে অবশ্যই থাকতে হবে, - মানে থাকা যাবে না — Google-স্টাইল সার্চ অপারেটর।

JSON কলাম কুয়েরি

কোড	ব্যাখ্যা
SELECT * FROM plans WHERE JSON_CONTAINS(features, '"podcast_access"')	JSON array-এর ভেতরে একটা ভ্যালু আছে কিনা
SELECT title, JSON_EXTRACT(condition_logic, '\$.limit') AS lim FROM paypal_rules	JSON অবজেক্টের নির্দিষ্ট key-এর ভ্যালু বের করা (-> শর্টহ্যান্ড আছে: condition_logic->'\$.limit')
UPDATE articles SET geo_restriction = JSON_ARRAY_APPEND(geo_restriction, '\$', 'IN') WHERE id = ?	JSON array-তে নতুন এলিমেন্ট যোগ করা

সতর্কতা — সাধারণ ইনডেক্স JSON কলামে কাজ করে না। দরকার হলে "generated column" বানিয়ে (JSON থেকে virtual কলাম বের করে) তার উপর ইনডেক্স করতে হয়: ALTER TABLE plans ADD COLUMN has_podcast BOOLEAN GENERATED ALWAYS AS (JSON_CONTAINS(features, '"podcast_access"')) VIRTUAL, ADD INDEX(has_podcast) /

- প্রশ্ন: কেন paypal_rules.condition_logic JSON রাখা হলো কিন্তু articles.status সাধারণ ENUM রাখা হলো — দুটোই তো ভিন্ন ভিন্ন মান নিতে পারে, পার্থক্যটা কোথায়?

Partitioning ও Partition Pruning

কোটি কোটি রো-এর টেবিলে কুয়েরি দ্রুত রাখা

পার্টিশন ভেতরে ভেতরে কী করে

behavioral_metrics টেবিল RANGE BY (YEAR(recorded_at)*100+MONTH(recorded_at)) দিয়ে পার্টিশন করা — প্রতিটা মাসের ডেটা আলাদা "সাব-টেবিলে" থাকে, যদিও উপর থেকে একটাই টেবিল মনে হয়।

```
SELECT COUNT(*) FROM behavioral_metrics
WHERE recorded_at >= '2026-08-01'
AND recorded_at < '2026-09-01'
```

MySQL বুঝে যায় এই কুয়েরির উত্তর শুধু আগস্ট-২০২৬ পার্টিশনে আছে — বাকি ২০+ মাসের পার্টিশন সম্পূর্ণ স্কিপ করে দেয়। এটাই "partition pruning"। কিন্তু এটা কাজ করে শুধু যখন WHERE-এ পার্টিশন-কী কলাম (recorded_at) থাকে — WHERE article_id=? দিলে সব পার্টিশন চেক করতে হবে।

EXPLAIN PARTITIONS দিয়ে যাচাই

```
EXPLAIN PARTITIONS
SELECT * FROM read_histories
WHERE user_id=5 AND read_at >= '2026-08-01';
```

partitions কলামে শুধু "p202608" দেখালে pruning কাজ করছে (ভালো), সব পার্টিশনের নাম দেখালে pruning কাজ করছে না।

কেন **Partitioned** টেবিলে **FK** কাজ করে না

একটা FK চেক করতে হলে referenced টেবিলের নির্দিষ্ট পার্টিশনে দ্রুত পৌঁছানোর গ্যারান্টি দরকার, কিন্তু FK-এর সাথে partition-key সবসময় সম্পর্কিত না থাকায় MySQL এটা সাপোর্ট করে না। তাই read_histories, behavioral_metrics, activity_logs-এ FK নেই — referential integrity Eloquent দিয়ে অ্যান্নিকেশন লেয়ারে সামলানো হয়।

- প্রশ্ন: partition pruning কাজ না করলে ১০ কোটি রো-এর টেবিলে কী সমস্যা হবে?

Pagination — Offset vs Cursor

পেজিনেশনের দুই পদ্ধতি

Offset Pagination — সহজ কিন্তু স্কেলে সমস্যা

```
SELECT * FROM articles
WHERE status='published'
ORDER BY published_at DESC
LIMIT 20 OFFSET 10000
```

পেজ ৫০১ দেখাতে MySQL-কে প্রথমে ১০,০২০টা রো স্ক্যান করতে হয়, তারপর প্রথম ১০,০০০টা ফেলে দিয়ে পরের ২০টা রিটার্ন করে — OFFSET যত বড় হয় কুয়েরি তত ধীর হয়, কারণ MySQL-কে সব বাদ-দেওয়া রো-ও গুনতে হয়।

Cursor (Keyset) Pagination — স্কেলে দ্রুত

```
SELECT * FROM articles
WHERE status='published'
  AND published_at < ? -- আগের পেজের শেষ published_at
ORDER BY published_at DESC
LIMIT 20
```

OFFSET-এর বদলে "আগের পেজের শেষ আইটেমের পরে থেকে শুরু করে" — ইনডেক্স ব্যবহার করে সরাসরি সেই বিন্দুতে জাম্প করে। পেজ ৫ হোক বা পেজ ৫০০, গতি প্রায় একই থাকে। ইনফিনিট-স্কল ফিডের (হোমপেজ, নিউজফিড) জন্য আদর্শ।

টাই-ব্রেকার প্রয়োজন — একই **published_at** হলে

```
SELECT * FROM articles
WHERE status='published'
  AND (published_at, id) < (?, ?)
ORDER BY published_at DESC, id DESC
LIMIT 20
```

যদি দুইটা আর্টিকেল ঠিক একই সেকেন্ডে পাবলিশ হয়, শুধু published_at দিয়ে cursor বানাতে একটা রো হারিয়ে যেতে পারে বা ডুপ্লিকেট দেখাতে পারে — তাই id-কেও (ইউনিক, টাই-ব্রেকার হিসেবে) cursor-এ যোগ করা নিরাপদ প্র্যাকটিস।

- প্রশ্ন: *cursor pagination-এ সরাসরি "পেজ ৭-এ যাও" বলা কেন কঠিন?*

Transaction, Locking ও Isolation Levels

একসাথে একাধিক পরিবর্তন নিরাপদ রাখা

ACID — চারটা গ্যারান্টি

অক্ষর	গ্যারান্টি
Atomicity	সব হবে, নয়তো কিছুই হবে না
Consistency	ট্রানজ্যাকশনের আগে ও পরে ডেটা সবসময় বৈধ নিয়ম মেনে চলে (constraint ভাঙে না)
Isolation	একসাথে চলা একাধিক ট্রানজ্যাকশন একে অপরের অসম্পূর্ণ কাজ দেখে না
Durability	COMMIT হওয়ার পর ডেটা স্থায়ীভাবে সংরক্ষিত থাকে, সার্ভার ক্র্যাশ করলেও হারায় না

Transaction — মৌলিক ব্যবহার

```
START TRANSACTION;
UPDATE subscriptions SET status='active' WHERE id=?;
INSERT INTO subscription_events (...) VALUES (...);
INSERT INTO transactions (...) VALUES (...);
COMMIT;
-- কোনো ধাপে সমস্যা হলে: ROLLBACK;
```

Isolation Level — চারটা স্তর

লেভেল	আচরণ	সমস্যা এড়ায়
READ UNCOMMITTED	অন্য ট্রানজ্যাকশনের এখনো commit-না-হওয়া পরিবর্তনও দেখা যায় ("dirty read")	কিছুই না — সবচেয়ে দুর্বল
READ COMMITTED	শুধু commit হওয়া ডেটা দেখা যায়, কিন্তু একই ট্রানজ্যাকশনে দুইবার পড়লে ভিন্ন ফলাফল আসতে পারে	Dirty read
REPEATABLE READ (MySQL ডিফল্ট)	একই ট্রানজ্যাকশনের ভেতরে বারবার পড়লে সবসময় একই ফলাফল	Dirty read, Non-repeatable read
SERIALIZABLE	সবচেয়ে কড়া — ট্রানজ্যাকশনগুলো একে একে (sequentially) চলার মতো আচরণ করে	সব রেস কন্ডিশন, কিন্তু performance সবচেয়ে কম

MySQL/InnoDB-এর ডিফল্ট **REPEATABLE READ** — বেশিরভাগ অ্যাপ্লিকেশনের জন্য এটাই যথেষ্ট ভারসাম্য (নিরাপত্তা বনাম পারফরম্যান্স)। **financial transaction**-এর মতো অত্যন্ত **sensitive** অপারেশনে **SERIALIZABLE** বা এক্সপ্লিসিট **locking** (নিচে দেখো) ব্যবহার করা হয়।

Race Condition ও Pessimistic Locking

দুইজন এডিটর একই আর্টিকেল একসাথে সেভ করলে, যে শেষে সেভ করে সে আরেকজনের পরিবর্তন মুছে ফেলতে পারে (silent data loss) — একে **race condition** বলে।

```
START TRANSACTION;
SELECT * FROM ad_campaigns WHERE id=? FOR UPDATE;
-- spent_amount চেক করে বাজেটের মধ্যে থাকলে:
UPDATE ad_campaigns SET spent_amount=spent_amount+? WHERE id=?;
COMMIT;
```

FOR UPDATE দিয়ে রো-টা লক করা হয় — অন্য কোনো ট্রানজ্যাকশন এই রো একসাথে পড়তে/বদলাতে পারবে না যতক্ষণ না প্রথম ট্রানজ্যাকশন **COMMIT/ROLLBACK** হয়। **ad_campaigns.spent_amount**-এর মতো জায়গায় বাজেট ওভারস্পেন্ড এড়াতে জরুরি — যদি লক না

নেওয়া হয়, দুইটা সমান্তরাল রিকোয়েস্ট একই পুরনো `spent_amount` পড়ে ফেলতে পারে, দুটোই আপডেট করার পরে একটা আপডেট হারিয়ে যায় (`lost update`)।

Advisory/Table-based Lock — `editorial_locks` প্যাটার্ন

DB-এর ট্রানজ্যাকশনাল `FOR UPDATE` লক শুধু সংক্ষিপ্ত সময়ের জন্য (মিলিসেকেন্ড-সেকেন্ড) ভালো, কারণ লক ধরে রাখা মানে ট্রানজ্যাকশন খোলা রাখা — দীর্ঘ সময় (মিনিট-ঘণ্টা, একজন এডিটর আর্টিকেল এডিট করছে) ট্রানজ্যাকশন খোলা রাখা DB-এর জন্য ক্ষতিকর। তাই `editorial_locks` নামে আলাদা একটা "লক টেবিল" ব্যবহার করা হয়েছে — একজন এডিটর কাজ শুরু করলে এখানে একটা রো তৈরি হয়, অ্যাপ্লিকেশন কোড এটা চেক করে অন্য এডিটরকে আটকে দেয়, কোনো দীর্ঘ ট্রানজ্যাকশন খোলা রাখা লাগে না।

- প্রশ্ন: `FOR UPDATE` লক নেওয়ার পর `COMMIT/ROLLBACK` না করে ট্রানজ্যাকশন খোলা রাখলে (যেমন অ্যাপ্লিকেশন ক্র্যাশ করলে) কী সমস্যা হবে?
- প্রশ্ন: "`lost update`" সমস্যা কী, আর `FOR UPDATE` কীভাবে এটা প্রতিরোধ করে?

Normalization ও Denormalization

ডেটা কীভাবে সংগঠিত করবে — তত্ত্ব থেকে বাস্তব ট্রেড-অফ

Normal Form — সংক্ষেপে

ফর্ম	নিয়ম	উদাহরণ ভায়েলেশন
1NF	প্রতিটা কলামে একটামাত্র (atomic) মান	tags কলামে 'sports,cricket,bd' রাখা 1NF ভাঙে — এর বদলে article_tag pivot টেবিল দরকার
2NF	1NF + non-key কলাম পুরো primary key-এর উপর নির্ভরশীল হতে হবে	composite key-তে আংশিক নির্ভরতা থাকলে ভাঙে
3NF	2NF + non-key কলাম অন্য non-key কলামের উপর নির্ভরশীল হতে পারবে না	articles-এ category_name রাখা (category_id থাকা সত্ত্বেও) — transitive dependency

এই প্রজেক্টের বেশিরভাগ টেবিল 3NF মেনে ডিজাইন করা — categories, tags, topics আলাদা টেবিল, articles শুধু তাদের ID রেফারেন্স করে। কিন্তু কিছু জায়গায় ইচ্ছাকৃতভাবে normalization ভাঙা হয়েছে (denormalization) — পারফরম্যান্সের জন্য।

Denormalization — কেন ইচ্ছাকৃতভাবে নিয়ম ভাঙা হয়

উদাহরণ	Normalized হলে	Denormalized (এই প্রজেক্টে)
users.subscription_status	প্রতিবার subscriptions টেবিলে JOIN করে বের করতে হতো	সরাসরি users-এ cached কলাম, event listener দিয়ে sync
articles.view_count	প্রতিবার behavioral_metrics COUNT করতে হতো (পার্টিশনড, বড়)	articles-এ cached সংখ্যা, ঘটায় sync
tags.article_count	প্রতিবার article_tag COUNT করতে হতো	cached কলাম, রাতে ব্যাচ sync

Trade-off: denormalization পড়ার গতি বাড়ায়, কিন্তু ডেটা দুই জায়গায় থাকায় সিক্স-আউট অফ হওয়ার ঝুঁকি তৈরি করে — তাই সবসময় একটা নির্দিষ্ট sync মেকানিজম (event listener বা scheduled job) থাকা বাধ্যতামূলক।

- প্রশ্ন: normalized ডিজাইন কখন denormalize করা উচিত — কোন সিগন্যাল দেখলে বুঝবে?

View, Stored Procedure, Trigger

সংক্ষিপ্ত পরিচিতি

View — একটা কুয়েরিকে "ভার্চুয়াল টেবিল" বানানো

```
CREATE VIEW active_subscribers AS
SELECT u.id, u.email, s.plan_id, s.ends_at
FROM users u
JOIN subscriptions s ON u.id = s.user_id
WHERE s.status = 'active';

SELECT * FROM active_subscribers WHERE plan_id = 3;
```

View কোনো ডেটা সংরক্ষণ করে না — প্রতিবার ব্যবহারের সময় আভারলাইং টেবিল থেকে লাইভ ডেটা নিয়ে আসে। জটিল JOIN পুনর্ব্যবহারের জন্য উপকারী, সেনসিটিভ কলাম লুকিয়ে সিকিউরিটি লেয়ার হিসেবেও ব্যবহার করা যায়।

Stored Procedure — ডেটাবেজের ভেতরে লজিক

```
DELIMITER //
CREATE PROCEDURE ExpireOldSubscriptions()
BEGIN
    UPDATE subscriptions SET status='expired'
    WHERE status='active' AND ends_at < NOW();
END //
DELIMITER ;
```

এই প্রজেক্টে **stored procedure** এড়িয়ে চলা হয়েছে — বিজনেস লজিক **Laravel** অ্যাপ্লিকেশন কোডে রাখা হয়েছে। কারণ: (১) অ্যাপ্লিকেশন কোড ভার্সন-কন্ট্রোল/টেস্ট/রিভিউ করা সহজ, (২) একাধিক ডেভেলপার **Git** দিয়ে **collaborate** করলে **DB stored procedure** ট্রাক করা কঠিন, (৩) ভবিষ্যতে **microservice**-এ ভাগ করতে গেলে লজিক **DB**-তে আটকে থাকলে কঠিন হয়ে যায়।

Trigger — কোনো ইভেন্টে স্বয়ংক্রিয় প্রতিক্রিয়া

```
CREATE TRIGGER after_comment_insert
AFTER INSERT ON comments
FOR EACH ROW
UPDATE articles SET comment_count = comment_count + 1
WHERE id = NEW.article_id;
```

এই প্যাটার্নও (**denormalized counter** আপডেট) **trigger** দিয়ে না করে **Laravel Model Event/Observer** দিয়ে করা হয় — একই যুক্তি: অ্যাপ্লিকেশন লেয়ারে থাকলে টেস্ট করা সহজ, **DB** আর কোডের মধ্যে "hidden behavior" ছড়িয়ে থাকে না।

সংক্ষেপে — কখন **DB**-লেভেল লজিক ব্যবহার করবে

- **View:** রিপোর্টিং/অ্যানালিটিক্সের জন্য জটিল কুয়েরি পুনর্ব্যবহার করতে, নির্দিষ্ট রোলের জন্য সীমিত কলাম দেখাতে।
- **Trigger:** শুধুমাত্র **hard constraint enforce** করতে যা অ্যাপ্লিকেশন লেয়ারে গ্যারান্টি দেওয়া কঠিন।
- **Stored Procedure:** এই প্রজেক্টে এড়িয়ে চলা হয়েছে — বিজনেস লজিক সবসময় **Laravel**-এ।
- প্রশ্ন: **phpMyAdmin** দিয়ে সরাসরি একটা কন্স্টেন্ট ডিলিট করলে, **trigger** থাকলে **comment_count** ঠিক থাকবে, কিন্তু **Laravel Observer** দিয়ে করলে কী সমস্যা হতে পারে?

PART B

অনুশীলন — ১০টি স্তর, সহজ থেকে এক্সপার্ট

প্রতিটা প্রশ্নে চেষ্টা করো নিজে সমাধান লেখার, তারপর দেওয়া সমাধানের সাথে মিলিয়ে নাও। রঙিন ট্যাগ কঠিনতা বোঝায়: সবুজ=সহজ, কমলা=মাঝারি, লাল=কঠিন, বেগুনি=এক্সপার্ট।

মডিউল **B1**

বেসিক **SELECT, WHERE, ORDER BY, LIMIT**

*ভিত্তি মজবুত করো — এই স্তর দ্রুত পার হওয়া উচিত***অনুশীলন 1.1.** সব পাবলিশড আর্টিকেল [সহজ]

সব প্রকাশিত আর্টিকেলের headline ও published_at দেখাও।

সমাধান:

```
SELECT headline, published_at FROM articles
WHERE status = 'published';
```

ব্যাখ্যা: সাধারণ SELECT + WHERE — কলাম নির্দিষ্ট করে আনা, শুধু status='published' শর্ত মেলা রো ফিল্টার করা।

অনুশীলন 1.2. ব্রেকিং নিউজ, নতুন আগে [সহজ]

যত ব্রেকিং নিউজ আছে, সবচেয়ে নতুন প্রথমে এমনভাবে দেখাও।

সমাধান:

```
SELECT headline, published_at FROM articles
WHERE is_breaking = 1
ORDER BY published_at DESC;
```

ব্যাখ্যা: ORDER BY ... DESC দিয়ে নতুন-থেকে-পুরনো সাজানো (DESC = descending)।

অনুশীলন 1.3. নির্দিষ্ট রেঞ্জের দাম [সহজ]

যেসব প্লানের দাম ৫০০ থেকে ১৫০০ টাকার মধ্যে সেগুলো দেখাও।

সমাধান:

```
SELECT name, price FROM plans
WHERE price BETWEEN 500 AND 1500;
```

ব্যাখ্যা: BETWEEN দুই প্রান্ত সহই (inclusive) কাজ করে — 500 ও 1500 দুটোই ফলাফলে থাকবে।

অনুশীলন 1.4. তিনটা নির্দিষ্ট স্ট্যাটাস [সহজ]

draft, pitch অথবা in_review — এই তিন স্ট্যাটাসের যেকোনো একটাতে থাকা আর্টিকেল দেখাও।

সমাধান:

```
SELECT headline, status FROM articles
WHERE status IN ('draft', 'pitch', 'in_review');
```

ব্যাখ্যা: IN একাধিক OR-কে সংক্ষেপে লেখার উপায়: status='draft' OR status='pitch' OR status='in_review' এর সমতুল্য।

অনুশীলন 1.5. জিমেইল ইউজার খোঁজা [সহজ]

যেসব ইউজারের ইমেইল @gmail.com দিয়ে শেষ হয় তাদের নাম ও ইমেইল দেখাও।

সমাধান:

```
SELECT first_name, email FROM users
WHERE email LIKE '%@gmail.com';
```

ব্যাখ্যা: % ওয়াইল্ডকার্ড দিয়ে "শেষে এটা থাকতে হবে" বোঝানো হয়েছে।

অনুশীলন 1.6. ফোন নাম্বার নেই এমন ইউজার [সহজ]

যেসব ইউজার ফোন নাম্বার দেননি তাদের খুঁজে বের করো।

সমাধান:

```
SELECT first_name, email FROM users
WHERE phone IS NULL;
```

ব্যাখ্যা: NULL চেক করতে সবসময় IS NULL ব্যবহার করতে হয়, = NULL কখনো কাজ করে না।

অনুশীলন 1.7. পেজিনেশন — তৃতীয় পেজ [সহজ]

সব প্রকাশিত আর্টিকেল নতুন-থেকে-পুরনো সাজিয়ে, ২০-প্রতি-পেজ ধরে তৃতীয় পেজের ডেটা আনো।

সমাধান:

```
SELECT * FROM articles
WHERE status='published'
ORDER BY published_at DESC
LIMIT 20 OFFSET 40;
```

ব্যাখ্যা: পেজ ৩ মানে প্রথম দুই পেজ (৪০টা রো) বাদ দিয়ে পরের ২০টা — $OFFSET = (পেজ_নাম্বার-১) \times প্রতি_পেজ$ ।

অনুশীলন 1.8. একাধিক শর্ত + বন্ধনী [মাঝারি]

প্রকাশিত এবং (ব্রেকিং অথবা ডেভেলপিং-স্টোরি) এমন আর্টিকেল দেখাও।

সমাধান:

```
SELECT headline FROM articles
WHERE status='published'
AND (is_breaking=1 OR is_developing_story=1);
```

ব্যাখ্যা: বন্ধনী ছাড়া AND সবসময় OR-এর আগে evaluate হয়, তাই লজিক ভুল হয়ে যেত — এখানে বন্ধনী জরুরি ছিল।

অনুশীলন 1.9. DISTINCT দিয়ে ইউনিক লোকেল [সহজ]

articles টেবিলে কতগুলো ভিন্ন locale ব্যবহার হয়েছে তার তালিকা দেখাও।

সমাধান:

```
SELECT DISTINCT locale FROM articles;
```

ব্যাখ্যা: DISTINCT ডুপ্লিকেট মান বাদ দিয়ে শুধু ইউনিক মানগুলো দেখায়।

অনুশীলন 1.10. নির্দিষ্ট দৈর্ঘ্যের বেশি হেডলাইন [মাঝারি]

যেসব আর্টিকেলের headline ৮০ ক্যারেক্টরের বেশি লম্বা, সেগুলোর headline ও দৈর্ঘ্য দেখাও।

সমাধান:

```
SELECT headline, CHAR_LENGTH(headline) AS len
FROM articles
WHERE CHAR_LENGTH(headline) > 80;
```

ব্যাখ্যা: CHAR_LENGTH() ব্যবহার করা হয়েছে (LENGTH() না) কারণ বাংলা টেক্সটে প্রতি অক্ষর একাধিক বাইট নেয়, আমরা অক্ষর সংখ্যা চাই বাইট সংখ্যা না।

অনুশীলন 1.11. NOT IN দিয়ে বাদ দেওয়া [মাঝারি]

draft ও archived ছাড়া বাকি সব স্ট্যাটাসের আর্টিকেল দেখাও।

সমাধান:

```
SELECT headline, status FROM articles
WHERE status NOT IN ('draft', 'archived');
```

ব্যাখ্যা: **NOT IN** তালিকায় থাকা মানগুলো বাদ দিয়ে বাকি সব রাখে — সতর্কতা: এই তালিকায় কোনো **NULL** থাকলে সমস্যা হতে পারে (অধ্যায় ২ দেখো)।

অনুশীলন 1.12. জটিল ফিল্টার + সাজানো **[কঠিন]**

বাংলা ভাষার, পেওয়াল-ছাড়া, প্রকাশিত আর্টিকেল — ভিউ-কাউন্ট বেশি থেকে কম সাজিয়ে প্রথম ১০টা দেখাও।

সমাধান:

```
SELECT headline, view_count FROM articles
WHERE locale='bn' AND is_paywalled=0 AND status='published'
ORDER BY view_count DESC
LIMIT 10;
```

ব্যাখ্যা: তিনটা শর্ত **AND** দিয়ে একসাথে, তারপর সাজানো ও সীমিত করা — বাস্তব-জীবনের "top-10 free reads" ফিচারের ভিত্তি।

মডিউল **B2**

JOIN অনুশীলন

একাধিক টেবিল সংযুক্ত করার চর্চা

অনুশীলন 2.1. আর্টিকেল ও তার ক্যাটাগরি [সহজ]

প্রতিটা আর্টিকেলের **headline**-এর পাশে তার প্রধান ক্যাটাগরির নাম দেখাও।

সমাধান:

```
SELECT a.headline, c.name AS category
FROM articles a
JOIN categories c ON a.primary_category_id = c.id;
```

ব্যাখ্যা: সাধারণ INNER JOIN — **primary_category_id** দিয়ে **categories** টেবিলের সাথে মেলানো।

অনুশীলন 2.2. কমেন্ট-শূন্য আর্টিকেলসহ [মাঝারি]

প্রতিটা আর্টিকেলের **headline** ও মোট কমেন্ট সংখ্যা দেখাও, একটাও কমেন্ট না থাকা আর্টিকেলও যেন তালিকায় থাকে (কাউন্ট ০)।

সমাধান:

```
SELECT a.headline, COUNT(c.id) AS total_comments
FROM articles a
LEFT JOIN comments c ON a.id = c.article_id
GROUP BY a.id;
```

ব্যাখ্যা: LEFT JOIN ব্যবহার করা হয়েছে যাতে কমেন্ট-শূন্য আর্টিকেলও বাদ না পড়ে — COUNT(c.id) NULL-এর জন্য ০ গণনা করে (COUNT(*) হলে ভুল হতো, ১ গণনা করত)।

অনুশীলন 2.3. সাব-ক্যাটাগরি সহ প্যারেন্ট [মাঝারি]

প্রতিটা সাব-ক্যাটাগরির নামের পাশে তার প্যারেন্ট ক্যাটাগরির নাম দেখাও (top-level ক্যাটাগরিও দেখাবে, প্যারেন্ট NULL সহ)।

সমাধান:

```
SELECT child.name AS sub_category, parent.name AS main_category
FROM categories child
LEFT JOIN categories parent ON child.parent_id = parent.id;
```

ব্যাখ্যা: Self-JOIN — একই টেবিলকে দুইবার আলাদা alias (child, parent) দিয়ে ব্যবহার করা হয়েছে। LEFT JOIN কারণ top-level ক্যাটাগরির parent_id NULL, তাদেরও দেখাতে হবে।

অনুশীলন 2.4. তিন-টেবিল JOIN [কঠিন]

প্রতিটা প্রকাশিত আর্টিকেলের **headline**, প্রধান লেখকের নাম এবং ক্যাটাগরির নাম একসাথে দেখাও।

সমাধান:

```
SELECT a.headline, u.first_name, c.name AS category
FROM articles a
JOIN article_author aa ON a.id = aa.article_id AND aa.is_primary=1
JOIN users u ON aa.user_id = u.id
JOIN categories c ON a.primary_category_id = c.id
WHERE a.status='published';
```

ব্যাখ্যা: তিনটা JOIN চেইন করা হয়েছে — article_author-এ aa.is_primary=1 শর্ত ON ক্লজেই দেওয়া হয়েছে, যাতে শুধু প্রধান লেখক পাওয়া যায় (co-author বাদ)।

অনুশীলন 2.5. ফ্রিল্যান্স চুক্তি নেই এমন সাংবাদিক **[সাবারি]**

যেসব `is_journalist=1` ইউজারের কোনো `freelancer_contracts` নেই তাদের খুঁজে বের করো (তারা হয়তো ফুল-টাইম কর্মচারী)।

সমাধান:

```
SELECT u.first_name, u.email
FROM users u
LEFT JOIN freelancer_contracts fc ON u.id = fc.user_id
WHERE u.is_journalist=1 AND fc.id IS NULL;
```

ব্যাখ্যা: এটা "Anti-JOIN" প্যাটার্ন — LEFT JOIN করে তারপর WHERE ...IS NULL দিয়ে ডান পাশে মিল-না-পাওয়া রো বেছে নেওয়া, অর্থাৎ "যাদের সম্পর্কিত রেকর্ড নেই"।

অনুশীলন 2.6. পলিমরফিক JOIN **[কঠিন]**

`article_id 42`-এ যুক্ত সব মিডিয়ার (mediables দিয়ে) ফাইলের নাম দেখাও।

সমাধান:

```
SELECT ma.original_filename, m.collection_name
FROM mediables m
JOIN media_assets ma ON m.media_id = ma.id
WHERE m.mediaable_type = 'Article' AND m.mediaable_id = 42;
```

ব্যাখ্যা: পলিমরফিক টেবিলে JOIN করার সময় `mediaable_type` চেক করাও জরুরি (শুধু `mediaable_id` দিয়ে না, কারণ ভিন্ন মডেলের একই ID থাকতে পারে)।

অনুশীলন 2.7. একাধিক পিভট একসাথে **[কঠিন]**

একটা আর্টিকেলের `headline`, তার সব ট্যাগের নাম (কমা দিয়ে জোড়া) এবং প্রধান ক্যাটাগরি দেখাও।

সমাধান:

```
SELECT a.headline, c.name AS category,
       GROUP_CONCAT(t.name SEPARATOR ', ') AS tags
FROM articles a
JOIN categories c ON a.primary_category_id=c.id
LEFT JOIN article_tag at2 ON a.id=at2.article_id
LEFT JOIN tags t ON at2.tag_id=t.id
WHERE a.id = 42
GROUP BY a.id;
```

ব্যাখ্যা: `GROUP_CONCAT()` একাধিক রো-এর মান একটা স্ট্রিং-এ জোড়া লাগায় — many-to-many সম্পর্ক এক-রো-তে "ফ্ল্যাট" করে দেখানোর সাধারণ কৌশল।

অনুশীলন 2.8. ইনস্টিটিউশনাল ডোমেইন যাচাই **[সাবারি]**

একজন ইউজারের ইমেইল ডোমেইন (`@du.ac.bd`) কোনো `institutional_domains`-এ নিবন্ধিত কিনা তা `institutional_accounts`-এর নামসহ দেখাও।

সমাধান:

```
SELECT ia.organization_name
FROM institutional_domains idom
JOIN institutional_accounts ia ON idom.institution_id = ia.id
WHERE idom.domain_name = 'du.ac.bd';
```

ব্যাখ্যা: নতুন ইউজার সাইনআপের সময় তার ইমেইলের ডোমেইন অংশ বের করে এই কুয়েরি দিয়ে চেক করা হয়, `institutional access auto-grant` করার জন্য।

অনুশীলন 2.9. পডকাস্ট ও তার হোস্ট **[সহজ]**

প্রতিটা পডকাস্টের নামের পাশে তার হোস্টের নাম দেখাও।

সমাধান:

```
SELECT p.title, u.first_name AS host
FROM podcasts p
JOIN users u ON p.host_id = u.id;
```

ব্যাখ্যা: সাধারণ INNER JOIN — host_id দিয়ে users-এর সাথে মেলানো।

অনুশীলন **2.10.** সেলফ-জয়েন দিয়ে **related articles** [কঠিন]

article_id 10-এর related articles-এর headline দেখাও, relation_type সহ, সবচেয়ে নতুন related আর্টিকেল আগে।

সমাধান:

```
SELECT a2.headline, ar.relation_type
FROM article_related ar
JOIN articles a2 ON ar.related_article_id = a2.id
WHERE ar.article_id = 10
ORDER BY a2.published_at DESC;
```

ব্যাখ্যা: article_related নিজেই articles-কে দুইবার রেফারেন্স করে (article_id, related_article_id) — এখানে শুধু related_article_id-এর দিকে JOIN করা হয়েছে বিস্তারিত দেখানোর জন্য।

অনুশীলন **2.11.** কমেন্টের রিপ্লাই গণনা [মাঝারি]

প্রতিটা top-level কমেন্টের (parent_id IS NULL) পাশে তার সরাসরি রিপ্লাই সংখ্যা দেখাও।

সমাধান:

```
SELECT c1.body, COUNT(c2.id) AS reply_count
FROM comments c1
LEFT JOIN comments c2 ON c2.parent_id = c1.id
WHERE c1.parent_id IS NULL
GROUP BY c1.id;
```

ব্যাখ্যা: Self-JOIN + LEFT JOIN + GROUP BY একসাথে — c1 প্যারেন্ট কমেন্ট, c2 তার সরাসরি চাইল্ড (এক লেভেল রিপ্লাই)।

মডিউল B3

Aggregation ও HAVING অনুশীলন

গণনা, যোগফল, গড় এবং গ্রুপ ফিল্টারিং

অনুশীলন 3.1. মোট সাবস্ক্রাইবার [সহজ]

কতজন ইউজারের subscription_status = active তা গণনা করো।

সমাধান:

```
SELECT COUNT(*) AS active_subscribers
FROM users
WHERE subscription_status = 'active';
```

ব্যাখ্যা: সাধারণ COUNT(*) — শুধু ফিল্টার হওয়া রো সংখ্যা।

অনুশীলন 3.2. প্রতি ক্যাটাগরিতে আর্টিকেল [সহজ]

প্রতিটা ক্যাটাগরিতে কয়টা প্রকাশিত আর্টিকেল আছে, বেশি থেকে কম সাজিয়ে দেখাও।

সমাধান:

```
SELECT primary_category_id, COUNT(*) AS total
FROM articles
WHERE status='published'
GROUP BY primary_category_id
ORDER BY total DESC;
```

ব্যাখ্যা: WHERE (রো ফিল্টার) → GROUP BY (গ্রুপ তৈরি) → ORDER BY (গ্রুপের ফলাফল সাজানো) — এক্সিকিউশন অর্ডার মনে রাখো।

অনুশীলন 3.3. গড় পড়ার সময় [মাঝারি]

প্রতিটা আর্টিকেলের গড় dwell_time_seconds (behavioral_metrics থেকে) দেখাও, শুধু যেসব আর্টিকলে ১০০-র বেশি সেশন রেকর্ড আছে।

সমাধান:

```
SELECT article_id, AVG(dwell_time_seconds) AS avg_dwell,
COUNT(*) AS sessions
FROM behavioral_metrics
GROUP BY article_id
HAVING COUNT(*) > 100;
```

ব্যাখ্যা: HAVING ব্যবহার করা হয়েছে কারণ ফিল্টার শর্তটা (সেশন সংখ্যা) একটা aggregate function (COUNT) এর উপর — WHERE-এ এটা লেখা যেত না।

অনুশীলন 3.4. সর্বোচ্চ আয় করা লেখক [মাঝারি]

কোন ৫ জন ফ্রিল্যান্সারের মোট ইনভয়েন্স এমাইন্ট সবচেয়ে বেশি তাদের নাম ও মোট এমাইন্ট দেখাও।

সমাধান:

```
SELECT u.first_name, SUM(fi.total_amount) AS total_earned
FROM freelancer_invoices fi
JOIN users u ON fi.user_id = u.id
WHERE fi.status = 'paid'
GROUP BY u.id
ORDER BY total_earned DESC
LIMIT 5;
```


ব্যাখ্যা: JOIN + SUM + GROUP BY + ORDER BY + LIMIT — একসাথে একটা সম্পূর্ণ "টপ-৫" রিপোর্ট তৈরি করার প্যাটার্ন।

অনুশীলন 3.5. একাধিক কলামে **GROUP BY** [কঠিন]

প্রতিটা (ক্যাটাগরি, ভাষা) জোড়ার জন্য কতগুলো আর্টিকেল আছে দেখাও।

সমাধান:

```
SELECT primary_category_id, locale, COUNT(*) AS total
FROM articles
GROUP BY primary_category_id, locale
ORDER BY primary_category_id, total DESC;
```

ব্যাখ্যা: দুইটা কলাম দিয়ে GROUP BY করলে প্রতিটা ইউনিক জোড়ার জন্য আলাদা গ্রুপ তৈরি হয় — এখানে "খেলা+বাংলা" আর "খেলা+ইংরেজি" আলাদা গ্রুপ।

অনুশীলন 3.6. কনভার্সন রেট গণনা [মাঝারি]

ad_analytics থেকে প্রতিটা ক্যাম্পেইনের মোট impression, click এবং overall CTR (click/impression) গণনা করো।

সমাধান:

```
SELECT ad_campaign_id,
       SUM(total_impressions) AS impressions,
       SUM(total_clicks) AS clicks,
       SUM(total_clicks)/SUM(total_impressions) AS ctr
FROM ad_analytics
GROUP BY ad_campaign_id;
```

ব্যাখ্যা: লক্ষণীয় — CTR এখানে SUM(clicks)/SUM(impressions) হিসেবে গণনা হয়েছে, AVG(daily_ctr) না — কারণ প্রতিদিনের CTR-এর গড় নেওয়া ভুল (দিনভেদে impression সংখ্যা ভিন্ন হতে পারে, weighted average দরকার)।

অনুশীলন 3.7. একাধিক শর্তের **conditional COUNT** [কঠিন]

প্রতিটা ক্যাটাগরিতে বাংলা ও ইংরেজি আর্টিকেলের সংখ্যা আলাদা কলামে (পাশাপাশি) দেখাও।

সমাধান:

```
SELECT primary_category_id,
       SUM(CASE WHEN locale='bn' THEN 1 ELSE 0 END) AS bangla,
       SUM(CASE WHEN locale='en' THEN 1 ELSE 0 END) AS english
FROM articles
GROUP BY primary_category_id;
```

ব্যাখ্যা: Conditional aggregation (pivot প্যাটার্ন, অধ্যায় ১২ দেখো) — একটা কুয়েরিতেই দুই ভাষার আলাদা কলাম তৈরি করা হয়েছে।

অনুশীলন 3.8. **WHERE** ও **HAVING** একসাথে [কঠিন]

গত ৭ দিনে যেসব ইউজার অন্তত ৩টা ভিন্ন আর্টিকেল পড়েছে তাদের user_id ও পঠিত-আর্টিকেল-সংখ্যা দেখাও।

সমাধান:

```
SELECT user_id, COUNT(DISTINCT article_id) AS reads
FROM read_histories
WHERE read_at >= NOW() - INTERVAL 7 DAY
GROUP BY user_id
HAVING COUNT(DISTINCT article_id) >= 3;
```

ব্যাখ্যা: WHERE দিয়ে সময়সীমা ফিল্টার (রো-লেভেল), HAVING দিয়ে গণনা ফিল্টার (গ্রুপ-লেভেল) — দুটোর কাজ ভিন্ন, তাই দুটোই দরকার।

অনুশীলন 3.9. সর্বোচ্চ ও সর্বনিম্ন একসাথে [মাঝারি]

প্রতিটা পাজলের সর্বোচ্চ ও সর্বনিম্ন সম্পন্ন-হওয়া সময় (`time_spent_seconds`) দেখাও, শুধু যারা সম্পন্ন করেছে তাদের নিয়ে।

সমাধান:

```
SELECT puzzle_id,
       MAX(time_spent_seconds) AS slowest,
       MIN(time_spent_seconds) AS fastest
FROM user_puzzle_sessions
WHERE is_completed = 1
GROUP BY puzzle_id;
```

ব্যাখ্যা: MAX ও MIN একই SELECT-এ একসাথে ব্যবহার করা যায় — দুটোই **aggregate function**, GROUP BY একই গ্রুপে দুটোর জন্যই প্রযোজ্য।

অনুশীলন 3.10. নেস্টেড **aggregation** [এক্সপার্ট]

প্রতিটা ক্যাটাগরিতে গড়ে কতগুলো আর্টিকলে কमेंট আছে বনাম মোট আর্টিকেল — একটা "engagement ratio" বের করো।

সমাধান:

```
SELECT category_totals.primary_category_id,
       category_totals.total_articles,
       commented.articles_with_comments,
       ROUND(commented.articles_with_comments / category_totals.total_articles,
2) AS ratio
FROM (
  SELECT primary_category_id, COUNT(*) AS total_articles
  FROM articles WHERE status='published'
  GROUP BY primary_category_id
) category_totals
JOIN (
  SELECT a.primary_category_id, COUNT(DISTINCT a.id) AS
articles_with_comments
  FROM articles a JOIN comments c ON a.id=c.article_id
  GROUP BY a.primary_category_id
) commented ON category_totals.primary_category_id =
commented.primary_category_id;
```

ব্যাখ্যা: দুইটা আলাদা **aggregation (derived table)** হিসেবে তৈরি করে তারপর তাদের JOIN করা হয়েছে — একটা কুয়েরিতে দুই ভিন্ন GROUP BY মেশানো সরাসরি সম্ভব না, তাই আলাদা করে তারপর জোড়া লাগাতে হয়েছে।

মডিউল **B4**

Subquery ও CTE অনুশীলন

কুয়েরির ভেতরে কুয়েরি — ধাপে ধাপে জটিল লজিক

অনুশীলন 4.1. IN subquery [মাঝারি]

যেসব ইউজারের অন্তত একটা active সাবস্ক্রিপশন আছে তাদের ইমেইল দেখাও (subquery ব্যবহার করে, JOIN না)।

সমাধান:

```
SELECT email FROM users
WHERE id IN (
  SELECT user_id FROM subscriptions WHERE status='active'
);
```

ব্যাখ্যা: ভেতরের subquery প্রথমে active সাবস্ক্রিপশনের user_id-গুলোর তালিকা বানায়, বাইরের কুয়েরি সেই তালিকার সাথে মেলায়।

অনুশীলন 4.2. EXISTS দিয়ে একই কাজ [মাঝারি]

উপরের একই প্রশ্ন এবার EXISTS দিয়ে সমাধান করো।

সমাধান:

```
SELECT email FROM users u
WHERE EXISTS (
  SELECT 1 FROM subscriptions s
  WHERE s.user_id = u.id AND s.status='active'
);
```

ব্যাখ্যা: EXISTS একটা correlated subquery — প্রতিটা ইউজারের জন্য চেক করে "কোনো active সাবস্ক্রিপশন আছে কিনা", প্রথম ম্যাচেই থেমে যায়।

অনুশীলন 4.3. Correlated subquery দিয়ে তুলনা [কঠিন]

যেসব আর্টিকেলের view_count তার নিজের ক্যাটাগরির গড় ভিউ-কাউন্টের চেয়ে বেশি সেগুলো দেখাও।

সমাধান:

```
SELECT a.headline, a.view_count
FROM articles a
WHERE a.view_count > (
  SELECT AVG(a2.view_count)
  FROM articles a2
  WHERE a2.primary_category_id = a.primary_category_id
);
```

ব্যাখ্যা: ভেতরের subquery বাইরের a.primary_category_id ব্যবহার করছে (correlated) — প্রতিটা আর্টিকেলের জন্য তার নিজ ক্যাটাগরির গড় নতুন করে হিসাব হয়।

অনুশীলন 4.4. Scalar subquery SELECT-এ [মাঝারি]

প্রতিটা আর্টিকেলের পাশে তার লেখকের মোট প্রকাশিত আর্টিকেল সংখ্যা (একজন প্রলিফিক লেখক কিনা বোঝাতে) দেখাও।

সমাধান:

```
SELECT a.headline, a.created_by,
  (SELECT COUNT(*) FROM articles a2
   WHERE a2.created_by = a.created_by AND a2.status='published') AS
author_total
FROM articles a
```

```
WHERE a.status = 'published';
```

ব্যাখ্যা: সতর্কতা: এই কুয়েরি বড় ডেটাসেটে ধীর হতে পারে কারণ প্রতিটা রো-এর জন্য **subquery** নতুন করে চলে — বাস্তবে এটা **window function** `COUNT(*) OVER(PARTITION BY created_by)` দিয়ে অনেক দ্রুত করা যায় (অধ্যায় ১০)।

অনুশীলন 4.5. NOT EXISTS — anti-pattern চেক [কঠিন]

যেসব প্রকাশিত আর্টিকলে একটাও `article_embeddings` নেই (এখনো এমবেড হয়নি) সেগুলো খুঁজে বের করো।

সমাধান:

```
SELECT a.id, a.headline
FROM articles a
WHERE a.status='published'
AND NOT EXISTS (
  SELECT 1 FROM article_embeddings ae WHERE ae.article_id = a.id
);
```

ব্যাখ্যা: NOT EXISTS ব্যবহার করা হয়েছে NOT IN-এর বদলে — কারণ `article_embeddings.article_id`-এ কখনো NULL থাকার সম্ভাবনা না থাকলেও, এটা নিরাপদ, স্পষ্ট এবং ভালো অভ্যাস।

অনুশীলন 4.6. সাধারণ CTE [মাঝারি]

CTE ব্যবহার করে প্রথমে এই মাসের প্রকাশিত আর্টিকেল বের করো, তারপর তাদের মধ্যে টপ-৫ সবচেয়ে বেশি কमेंট পাওয়া আর্টিকেল দেখাও।

সমাধান:

```
WITH this_month_articles AS (
  SELECT * FROM articles
  WHERE status='published'
  AND published_at >= DATE_FORMAT(NOW(), '%Y-%m-01')
)
SELECT headline, comment_count
FROM this_month_articles
ORDER BY comment_count DESC
LIMIT 5;
```

ব্যাখ্যা: CTE দিয়ে প্রথম ধাপ (এই মাসের আর্টিকেল বাছাই) আলাদা করে নাম দেওয়া হয়েছে, তারপর সেটাকে সাধারণ টেবিলের মতো ব্যবহার করা হয়েছে।

অনুশীলন 4.7. একাধিক CTE চেইন [কঠিন]

CTE দিয়ে ধাপে ধাপে বের করো: (১) প্রতিটা ক্যাটাগরির মোট আর্টিকেল, (২) তাদের মধ্যে গড়ের চেয়ে বেশি আর্টিকেল আছে এমন ক্যাটাগরি।

সমাধান:

```
WITH category_counts AS (
  SELECT primary_category_id, COUNT(*) AS total
  FROM articles WHERE status='published'
  GROUP BY primary_category_id
),
overall_avg AS (
  SELECT AVG(total) AS avg_total FROM category_counts
)
SELECT cc.primary_category_id, cc.total
FROM category_counts cc, overall_avg oa
WHERE cc.total > oa.avg_total;
```

ব্যাখ্যা: দ্বিতীয় CTE (`overall_avg`) প্রথম CTE-কেই (`category_counts`) ব্যবহার করছে — এভাবে ধাপে ধাপে জটিল লজিক বানানো যায়, প্রতিটা ধাপ আলাদাভাবে **readable**।

অনুশীলন 4.8. Recursive CTE — কমেন্ট থ্রেড [এক্সপার্ট]

Recursive CTE ব্যবহার করে একটা নির্দিষ্ট top-level কমেন্টের (id=5) নিচের সব রিপ্লাই (যেকোনো গভীরতা) বের করে, depth সহ।

সমাধান:

```
WITH RECURSIVE thread AS (
  SELECT id, parent_id, body, 0 AS depth
  FROM comments WHERE id = 5
  UNION ALL
  SELECT c.id, c.parent_id, c.body, t.depth + 1
  FROM comments c
  JOIN thread t ON c.parent_id = t.id
)
SELECT * FROM thread ORDER BY depth;
```

ব্যাখ্যা: Anchor অংশ (প্রথম SELECT) শুরুর কমেন্ট বেছে নেয়, recursive অংশ (দ্বিতীয় SELECT) বারবার নিজে থেকে (thread) রেফারেন্স করে গভীরে যায় যতক্ষণ না নতুন রিপ্লাই না পাওয়া যায়।

অনুশীলন 4.9. Derived table + JOIN [কঠিন]

প্রতিটা ক্যাটাগরির নাম এবং তার মোট আর্টিকেল সংখ্যা — derived table (subquery in FROM) ব্যবহার করে দেখাও।

সমাধান:

```
SELECT c.name, stats.total
FROM categories c
JOIN (
  SELECT primary_category_id, COUNT(*) AS total
  FROM articles GROUP BY primary_category_id
) stats ON c.id = stats.primary_category_id;
```

ব্যাখ্যা: derived table (stats) প্রথমে aggregation করে একটা "ভার্চুয়াল টেবিল" বানায়, তারপর সেটাকে categories-এর সাথে JOIN করা হয়।

অনুশীলন 4.10. Subquery দিয়ে সর্বোচ্চ খোঁজা [কঠিন]

প্রতিটা পডকাস্টের সবচেয়ে নতুন এপিসোডের title দেখাও (একটামাত্র, MAX ব্যবহার করে না)।

সমাধান:

```
SELECT p.title AS podcast, pe.title AS latest_episode
FROM podcasts p
JOIN podcast_episodes pe ON pe.podcast_id = p.id
WHERE pe.published_at = (
  SELECT MAX(pe2.published_at)
  FROM podcast_episodes pe2
  WHERE pe2.podcast_id = p.id
);
```

ব্যাখ্যা: Correlated subquery দিয়ে প্রতিটা পডকাস্টের সর্বোচ্চ published_at বের করে, তারপর সেই তারিখের এপিসোড JOIN করা হয়েছে — এই একই কাজ Window Function দিয়েও করা যায় (অধ্যায় ১০, ROW_NUMBER)।

মডিউল **B5**

Window Functions অনুশীলন

RANK, LAG/LEAD, running total — বাস্তব সমস্যায় প্রয়োগ

অনুশীলন 5.1. সাধারণ **RANK** [মাঝারি]

সব আর্টিকেলকে `view_count` অনুযায়ী র‍্যাংক দাও (বেশি ভিউ = ভালো র‍্যাংক)।

সমাধান:

```
SELECT headline, view_count,
       RANK() OVER (ORDER BY view_count DESC) AS rnk
FROM articles
WHERE status='published';
```

ব্যাখ্যা: `PARTITION BY` ছাড়া পুরো টেবিলকে একটা গ্রুপ ধরে র‍্যাংক দেওয়া হয়েছে।

অনুশীলন 5.2. প্রতি-গ্রুপে টপ-N [কঠিন]

প্রতিটা ক্যাটাগরির টপ-৩ সবচেয়ে বেশি ভিউ পাওয়া আর্টিকেল দেখাও।

সমাধান:

```
SELECT * FROM (
  SELECT headline, primary_category_id, view_count,
         ROW_NUMBER() OVER (
           PARTITION BY primary_category_id ORDER BY view_count DESC
         ) AS rn
  FROM articles WHERE status='published'
) ranked
WHERE rn <= 3;
```

ব্যাখ্যা: `PARTITION BY primary_category_id` দিয়ে প্রতিটা ক্যাটাগরি আলাদাভাবে র‍্যাংক করা হয়েছে — এটাই "প্রতি গ্রুপে টপ-N" প্যাটার্নের আদর্শ সমাধান, `GROUP BY` দিয়ে সরাসরি সম্ভব না।

অনুশীলন 5.3. দিনের পার্থক্য **LAG** দিয়ে [কঠিন]

একটা নির্দিষ্ট আর্টিকেলের (`id=42`) প্রতিদিনের ভিউ ও আগের দিনের তুলনায় পরিবর্তন দেখাও।

সমাধান:

```
SELECT snapshot_date, total_views,
       total_views - LAG(total_views) OVER (ORDER BY snapshot_date) AS change
FROM content_performance_snapshots
WHERE article_id = 42
ORDER BY snapshot_date;
```

ব্যাখ্যা: `LAG()` আগের রো-এর মান নিয়ে আসে — প্রথম দিনের জন্য `LAG NULL` হবে (আগের কোনো রো নেই), তাই `change`-ও `NULL` হবে।

অনুশীলন 5.4. র‍ানিং টোটাল [মাঝারি]

একটা ইনভয়েসের (`id=7`) লাইন-আইটেমগুলোর ক্রমবর্ধমান যোগফল (`running total`) দেখাও।

সমাধান:

```
SELECT description, amount,
       SUM(amount) OVER (ORDER BY id) AS running_total
FROM invoice_items
WHERE invoice_id = 7;
```

ব্যাখ্যা: ORDER BY ছাড়া SUM() OVER() পুরো পার্টিশনের মোট যোগফল দিত (স্থির মান), ORDER BY দিলে "এখন পর্যন্ত যোগফল" (cumulative) হয়ে যায় — এটাই ডিফল্ট frame আচরণ।

অনুশীলন 5.5. সাত-দিনের চলমান গড় [কঠিন]

একটা আর্টিকেলের (id=15) প্রতিদিনের ভিউয়ের ৭-দিনের rolling average দেখাও।

সমাধান:

```
SELECT snapshot_date, total_views,
       AVG(total_views) OVER (
         ORDER BY snapshot_date
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS rolling_avg
FROM content_performance_snapshots
WHERE article_id = 15;
```

ব্যাখ্যা: ROWS BETWEEN 6 PRECEDING AND CURRENT ROW মানে বর্তমান রো + আগের ৬টা রো = মোট ৭টা রো-এর গড়।

অনুশীলন 5.6. RANK vs DENSE_RANK ফারাক [এক্সপার্ট]

একই view_count থাকা আর্টিকেলগুলো (টাই) থাকলে RANK() ও DENSE_RANK() দুটোই একসাথে দেখিয়ে পার্থক্য যাচাই করো।

সমাধান:

```
SELECT headline, view_count,
       RANK() OVER (ORDER BY view_count DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY view_count DESC) AS dense_rnk
FROM articles
WHERE status='published'
ORDER BY view_count DESC
LIMIT 20;
```

ব্যাখ্যা: টাই থাকা রো-গুলোর পরে RANK() একটা গ্যাপ রাখে (পরের র্যাংক টাই-সংখ্যা অনুযায়ী লাফ দেয়), DENSE_RANK() গ্যাপ রাখে না — ফলাফল টেবিলের প্রথম ২০ সারিতে এই পার্থক্য চোখে পড়বে যদি টাই থাকে।

অনুশীলন 5.7. কোয়ার্টাইল বিভাজন [কঠিন]

NTILE(4) ব্যবহার করে ইউজারদের তাদের মোট পঠিত-আর্টিকেল-সংখ্যা অনুযায়ী ৪টা সমান দলে ভাগ করো (quartile 1 = সবচেয়ে বেশি পড়ুয়া)।

সমাধান:

```
SELECT user_id, total_reads,
       NTILE(4) OVER (ORDER BY total_reads DESC) AS quartile
FROM (
  SELECT user_id, COUNT(DISTINCT article_id) AS total_reads
  FROM read_histories GROUP BY user_id
) t;
```

ব্যাখ্যা: NTILE(4) প্রতিটা রো-কে ১-৪ এর মধ্যে একটা গ্রুপ নাম্বার দেয়, চেষ্টা করে গ্রুপগুলো যতটা সম্ভব সমান আকারের হয় — মার্কেটিং সেগমেন্টেশনে খুবই উপকারী।

অনুশীলন 5.8. একই কুয়েরিতে একাধিক window function [এক্সপার্ট]

প্রতিটা আর্টিকেলের জন্য একসাথে দেখাও: তার ক্যাটাগরিতে র্যাংক, ক্যাটাগরির গড় ভিউ-কাউন্ট (window দিয়ে), এবং তার নিজের ভিউ সেই গড়ের কতগুণ।

সমাধান:

```
SELECT headline, primary_category_id, view_count,
       RANK() OVER (PARTITION BY primary_category_id ORDER BY view_count DESC)
```

```
AS rnk,  
  AVG(view_count) OVER (PARTITION BY primary_category_id) AS cat_avg,  
  ROUND(view_count / AVG(view_count) OVER (PARTITION BY  
primary_category_id), 2) AS ratio_to_avg  
FROM articles  
WHERE status='published';
```

ব্যাখ্যা: একই SELECT-এ একাধিক window function ব্যবহার করা যায়, প্রতিটির আলাদা (বা একই) PARTITION BY থাকতে পারে — MySQL প্রতিটা আলাদাভাবে গণনা করে কিন্তু একবারই ডেটা স্ক্যান করে।

মডিউল B6

Set Operations ও CASE WHEN অনুশীলন*UNION, INTERSECT, EXCEPT ও কন্ডিশনাল লজিক*অনুশীলন **6.1. UNION** দিয়ে দুই সোর্স মেশানো [\[মারারি\]](#)

সব ইমেইল ঠিকানা একসাথে দেখাও — users টেবিল ও subscribers টেবিল থেকে, ডুপ্লিকেট বাদ দিয়ে।

সমাধান:

```
SELECT email FROM users WHERE email IS NOT NULL
UNION
SELECT email FROM subscribers;
```

ব্যাখ্যা: UNION (UNION ALL না) ব্যবহার করা হয়েছে কারণ একই ইমেইল দুই টেবিলেই থাকতে পারে (যে ইউজার নিউজলেটারও সাবস্ক্রাইব করেছে), আমরা ডুপ্লিকেট চাই না।

অনুশীলন **6.2. CASE WHEN** দিয়ে ক্যাটাগরাইজ করা [\[সহজ\]](#)

প্রতিটা আর্টিকেলের view_count অনুযায়ী viral/popular/normal/low — এই চারটা ট্যাগের একটা দাও।

সমাধান:

```
SELECT headline,
CASE
  WHEN view_count > 100000 THEN 'viral'
  WHEN view_count > 10000 THEN 'popular'
  WHEN view_count > 1000 THEN 'normal'
  ELSE 'low'
END AS tier
FROM articles;
```

ব্যাখ্যা: উপর থেকে নিচে প্রথম সত্য হওয়া শর্তের ফলাফল রিটার্ন হয়।

অনুশীলন **6.3. Conditional aggregation** দিয়ে ব্রেকডাউন [\[কঠিন\]](#)

প্রতিটা payment_gateway অনুযায়ী মোট transaction, সফল সংখ্যা এবং ব্যর্থ সংখ্যা আলাদা কলামে দেখাও।

সমাধান:

```
SELECT payment_gateway,
COUNT(*) AS total,
SUM(CASE WHEN status='completed' THEN 1 ELSE 0 END) AS successful,
SUM(CASE WHEN status='failed' THEN 1 ELSE 0 END) AS failed
FROM transactions
GROUP BY payment_gateway;
```

ব্যাখ্যা: পিভট-স্টাইল রিপোর্ট — একটা কুয়েরিতেই গেটওয়ে-ভিত্তিক সাফল্যের হার দেখা যাচ্ছে, একাধিক আলাদা কুয়েরি লাগছে না।

অনুশীলন **6.4. EXCEPT** দিয়ে নিষ্ক্রিয় খুঁজে বের করা [\[মারারি\]](#)

যেসব সাবস্ক্রাইবারের ইমেইল আছে কিন্তু গত ৩০ দিনে read_histories-এ কোনো activity নেই (user_id দিয়ে) তাদের খুঁজে বের করো।

সমাধান:

```
SELECT id FROM users WHERE subscription_status='active'
EXCEPT
SELECT DISTINCT user_id FROM read_histories
WHERE read_at >= NOW() - INTERVAL 30 DAY;
```

ব্যাখ্যা: **EXCEPT** প্রথম কুয়েরির ফলাফল থেকে দ্বিতীয় কুয়েরির ফলাফল বাদ দেয় — "সক্রিয় সাবস্ক্রাইবার কিন্তু সাম্প্রতিক পড়াশোনা নেই" (churn risk) চিহ্নিত করতে উপকারী।

অনুশীলন 6.5. CASE WHEN নেস্টেড শর্ত [\[কঠিন\]](#)

সাবস্ক্রিপশনের মেয়াদ শেষ হওয়ার তারিখ অনুযায়ী "expired" / "expiring_soon" (৭ দিনের মধ্যে) / "active" ট্যাগ দাও।

সমাধান:

```
SELECT id, ends_at,
       CASE
         WHEN ends_at < NOW() THEN 'expired'
         WHEN ends_at < NOW() + INTERVAL 7 DAY THEN 'expiring_soon'
         ELSE 'active'
       END AS status_tag
FROM subscriptions
WHERE status = 'active';
```

ব্যাখ্যা: তারিখ-ভিত্তিক তিন-স্তরের শ্রেণীবিভাগ — রিনিউয়াল রিমাইন্ডার পাঠানোর ফিচারের ভিত্তি হতে পারে এটা।

অনুশীলন 6.6. INTERSECT দিয়ে দুই আচরণ মেলানো [\[এক্সপার্ট\]](#)

যেসব ইউজার এই সপ্তাহে অন্তত একটা আর্টিকেল পড়েছে এবং একটা কমেন্টও করেছে (দুটোই সত্য) তাদের user_id দেখাও।

সমাধান:

```
SELECT DISTINCT user_id FROM read_histories
WHERE read_at >= NOW() - INTERVAL 7 DAY
INTERSECT
SELECT DISTINCT user_id FROM comments
WHERE created_at >= NOW() - INTERVAL 7 DAY;
```

ব্যাখ্যা: **INTERSECT** দুই কুয়েরির কমন অংশ রাখে — এখানে "পড়া" ও "কমেন্ট করা" দুই আচরণ যাদের মধ্যেই আছে তাদের বের করছে (highly-engaged ইউজার সেগমেন্ট)।

অনুশীলন 6.7. UNION ALL দিয়ে অ্যাক্টিভিটি টাইমলাইন [\[মাঝারি\]](#)

একটা ইউজারের (id=10) সব অ্যাক্টিভিটি — পড়া ও কমেন্ট — একটা মিলিত টাইমলাইনে (টাইপ ও সময়সহ) দেখাও।

সমাধান:

```
SELECT 'read' AS activity_type, read_at AS occurred_at FROM read_histories
WHERE user_id=10
UNION ALL
SELECT 'comment' AS activity_type, created_at AS occurred_at FROM comments
WHERE user_id=10
ORDER BY occurred_at DESC;
```

ব্যাখ্যা: **UNION ALL** ব্যবহার করা হয়েছে (**UNION** না) কারণ এখানে ডুপ্লিকেট থাকার প্রশ্নই নেই (**activity_type** কলামই আলাদা করে দিচ্ছে প্রতিটা রো), তাই **dedup**-এর বাড়তি খরচ এড়ানো হয়েছে।

মডিউল B7

String ও Date Function অনুশীলন

টেক্সট ও সময় নিয়ে কাজ করার হাতিয়ার

অনুশীলন 7.1. পূর্ণ নাম তৈরি [সহজ]

first_name ও last_name জোড়া দিয়ে একটা full_name কলাম দেখাও।

সমাধান:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM users;
```

ব্যাখ্যা: CONCAT() সরাসরি স্ট্রিং জোড়া লাগায়। সতর্কতা: যদি last_name NULL হয়, পুরো CONCAT ফলাফল NULL হয়ে যাবে (CONCAT_WS ব্যবহার করলে এই সমস্যা এড়ানো যায়)।

অনুশীলন 7.2. হেডলাইন সংক্ষিপ্ত করা [মাঝারি]

প্রতিটা আর্টিকেলের হেডলাইনের প্রথম ৫০ ক্যারেক্টার + "..." দেখাও যদি হেডলাইন ৫০ এর বেশি লম্বা হয়।

সমাধান:

```
SELECT CASE
  WHEN CHAR_LENGTH(headline) > 50
    THEN CONCAT(SUBSTRING(headline, 1, 50), '...')
  ELSE headline
END AS short_headline
FROM articles;
```

ব্যাখ্যা: SUBSTRING + CASE WHEN একসাথে — শুধু লম্বা হেডলাইনেই "..." যোগ হচ্ছে, ছোটগুলো অক্ষত থাকছে।

অনুশীলন 7.3. ইমেইল ডোমেইন বের করা [মাঝারি]

প্রতিটা ইউজারের ইমেইলের ডোমেইন অংশ (@-এর পরের অংশ) আলাদা করে দেখাও।

সমাধান:

```
SELECT email,
  SUBSTRING(email, LOCATE('@', email) + 1) AS domain
FROM users
WHERE email IS NOT NULL;
```

ব্যাখ্যা: LOCATE() @ চিহ্নের পজিশন বের করে, SUBSTRING সেখান থেকে বাকি অংশ কেটে নেয়।

অনুশীলন 7.4. বড় হাতের ক্যাটাগরি স্লাগ [সহজ]

সব ক্যাটাগরির slug বড় হাতের অক্ষরে (uppercase) দেখাও।

সমাধান:

```
SELECT UPPER(slug) FROM categories;
```

ব্যাখ্যা: UPPER() সহজ কেস-রূপান্তর।

অনুশীলন 7.5. ট্রিম করে তুলনা [মাঝারি]

search_histories-এ যেসব search_term-এর আশেপাশে অতিরিক্ত স্পেস আছে তা বাদ দিয়ে সবচেয়ে জনপ্রিয় ১০টা টার্ম দেখাও।

সমাধান:

```
SELECT TRIM(search_term) AS term, COUNT(*) AS cnt
```

```
FROM search_histories
GROUP BY TRIM(search_term)
ORDER BY cnt DESC LIMIT 10;
```

ব্যাখ্যা: TRIM() দিয়ে GROUP BY করলে " নির্বাচন" আর "নির্বাচন " একই গ্রুপ পড়বে, যা TRIM ছাড়া আলাদা গ্রুপ হয়ে যেত (ভুল ফলাফল দিত)।

অনুশীলন 7.6. আজকের প্রকাশনা [সহজ]

আজকে প্রকাশিত সব আর্টিকেল দেখাও।

সমাধান:

```
SELECT headline FROM articles
WHERE DATE(published_at) = CURDATE();
```

ব্যাখ্যা: DATE() published_at (যেখানে সময়ও আছে) থেকে শুধু তারিখের অংশ বের করে, CURDATE() দিয়ে আজকের তারিখের সাথে তুলনা।

অনুশীলন 7.7. বয়স গণনা (দিনে) [মাঝারি]

প্রতিটা প্রকাশিত আর্টিকেল কত দিন আগে প্রকাশিত হয়েছে তা দেখাও, নতুন থেকে পুরনো সাজিয়ে।

সমাধান:

```
SELECT headline, DATEDIFF(NOW(), published_at) AS days_old
FROM articles
WHERE status='published'
ORDER BY published_at DESC;
```

ব্যাখ্যা: DATEDIFF(তারিখ_১, তারিখ_২) সবসময় তারিখ_১ – তারিখ_২ (দিনে) রিটার্ন করে।

অনুশীলন 7.8. সপ্তাহের দিন অনুযায়ী গ্রুপিং [কঠিন]

গত ৩০ দিনে সপ্তাহের কোন দিনে সবচেয়ে বেশি আর্টিকেল প্রকাশিত হয়েছে দেখাও।

সমাধান:

```
SELECT DAYNAME(published_at) AS day_name, COUNT(*) AS total
FROM articles
WHERE published_at >= NOW() - INTERVAL 30 DAY
GROUP BY DAYNAME(published_at), DAYOFWEEK(published_at)
ORDER BY DAYOFWEEK(published_at);
```

ব্যাখ্যা: DAYNAME() পড়ার জন্য নাম দেখায় কিন্তু বর্ণানুক্রমিকভাবে সাজাবে (Friday, Monday...), তাই GROUP BY-তে DAYOFWEEK()-ও যোগ করে সেটা দিয়ে সাজানো হয়েছে (সঠিক সাপ্তাহিক ক্রমে)।

অনুশীলন 7.9. "এই মাসের" প্যাটার্ন [কঠিন]

এই মাসের মোট কতজন নতুন ইউজার রেজিস্টার করেছে তা গণনা করো।

সমাধান:

```
SELECT COUNT(*) AS new_users
FROM users
WHERE created_at >= DATE_FORMAT(NOW(), '%Y-%m-01');
```

ব্যাখ্যা: DATE_FORMAT(NOW(), '%Y-%m-01') এই মাসের ১ তারিখ বের করে — paywall metering-এও (অধ্যায় ১৪) এই একই প্যাটার্ন ব্যবহার হয়।

অনুশীলন 7.10. সাপ্তাহিক ট্রেন্ড রিপোর্ট [\[এক্সপার্ট\]](#)

গত ৮ সপ্তাহের প্রতিটার জন্য মোট নতুন সাবস্ক্রিপশন সংখ্যা দেখাও, সপ্তাহ-শুরুর তারিখসহ।

সমাধান:

```
SELECT
    DATE_SUB(DATE(created_at), INTERVAL WEEKDAY(created_at) DAY) AS
    week_start,
    COUNT(*) AS new_subscriptions
FROM subscriptions
WHERE created_at >= NOW() - INTERVAL 8 WEEK
GROUP BY week_start
ORDER BY week_start;
```

ব্যাখ্যা: WEEKDAY() সোমবার=0 থেকে রবিবার=6 রিটার্ন করে, DATE_SUB দিয়ে তাই সেই সপ্তাহের সোমবারে ফিরে যাওয়া হয়েছে — এটাই "সপ্তাহ-ভিত্তিক গ্রুপিং"-এর একটা প্রচলিত কৌশল।

মডিউল **B8**

NULL হান্ডলিং ও Index/EXPLAIN অনুশীলন

*সবচেয়ে সূক্ষ্ম কিন্তু গুরুত্বপূর্ণ বিষয়গুলো***অনুশীলন 8.1. COALESCE** দিয়ে ডিফল্ট মান [\[সহজ\]](#)

প্রতিটা আর্টিকেলের SEO title দেখাও — যদি seo_title না থাকে, headline দেখাও।

সমাধান:

```
SELECT COALESCE(seo_title, headline) AS display_title FROM articles;
```

ব্যাখ্যা: COALESCE বাম থেকে প্রথম non-NULL মান রিটার্ন করে — seo_title NULL হলে headline ব্যবহার হবে।

অনুশীলন 8.2. একাধিক ফলব্যাক [\[মাঝারি\]](#)

প্রতিটা আর্টিকেল-অথর জোড়ার বাইলাইন দেখাও — byline_override থাকলে সেটা, নাহলে ইউজারের first_name+last_name।

সমাধান:

```
SELECT COALESCE(aa.byline_override, CONCAT(u.first_name, ' ', u.last_name))
AS byline
FROM article_author aa
JOIN users u ON aa.user_id = u.id;
```

ব্যাখ্যা: COALESCE-এর প্রথম আর্গুমেন্ট override, দ্বিতীয়টা fallback হিসেবে CONCAT দিয়ে তৈরি করা পুরো নাম।

অনুশীলন 8.3. NULL-safe তুলনা [\[কঠিন\]](#)

দুইটা কলাম (device_fingerprint ও একটা তুলনামূলক ভ্যালু) তুলনা করো যেখানে উভয়েই NULL হতে পারে, স্বাভাবিক = অপারেটরের ভুল এড়িয়ে।

সমাধান:

```
SELECT * FROM article_reactions
WHERE device_fingerprint <=> NULL;
```

ব্যাখ্যা: <=> (NULL-safe equal) অপারেটর NULL-কেও একটা তুলনায়োগ্য মান হিসেবে ট্রিট করে — সাধারণ = কখনো NULL-এর সাথে true রিটার্ন করে না, কিন্তু <=> করে।

অনুশীলন 8.4. NULL বাদ দেওয়া বনাম রাখা [\[মাঝারি\]](#)

সব ইউজার দেখাও, phone না থাকলে "N/A" লেখা দেখাও (স্কিপ না করে)।

সমাধান:

```
SELECT first_name, COALESCE(phone, 'N/A') AS phone
FROM users;
```

ব্যাখ্যা: এখানে WHERE phone IS NOT NULL ব্যবহার করলে ভুল হতো (phone-না-থাকা ইউজার বাদ পড়ে যেত) — আমরা সবাইকে দেখাতে চাই, শুধু NULL-এর জায়গায় ডিফল্ট টেক্সট চাই।

অনুশীলন 8.5. EXPLAIN পড়া [\[কঠিন\]](#)

নিচের কুয়েরির EXPLAIN আউটপুট কল্লনা করো এবং বলো type কলামে কী আসবে যদি (status, published_at) কম্পোজিট ইনডেক্স থাকে: SELECT * FROM articles WHERE status='published' ORDER BY published_at DESC LIMIT 20;

সমাধান:

```
EXPLAIN SELECT * FROM articles
```

```
WHERE status='published'
ORDER BY published_at DESC LIMIT 20;
```

ব্যাখ্যা: যেহেতু ইনডেক্স (status, published_at) leftmost prefix rule মেনে চলে এবং WHERE কলাম (status) leftmost, type=ref (বা range) হবে, key কলামে ইনডেক্সের নাম দেখাবে, এবং Extra-তে "Using index condition" বা filesort ছাড়াই sorted আউটপুট পাওয়া যাবে কারণ ইনডেক্সই আগে থেকে sorted।

অনুশীলন 8.6. কোন ইনডেক্স কাজ করবে না তা চেনা [কঠিন]

নিচের দুইটা কুয়েরির মধ্যে কোনটা (status, published_at) কম্পোজিট ইনডেক্স ব্যবহার করতে পারবে না, আর কেন: (ক) WHERE published_at > ? (খ) WHERE status=? AND published_at > ?

সমাধান:

```
-- (ক) কাজ করবে না — leftmost কলাম (status) অনুপস্থিত
SELECT * FROM articles WHERE published_at > '2026-01-01';

-- (খ) কাজ করবে — leftmost কলাম আছে
SELECT * FROM articles WHERE status='published' AND published_at > '2026-01-01';
```

ব্যাখ্যা: Leftmost Prefix Rule (অধ্যায় ১৬) অনুযায়ী কম্পোজিট ইনডেক্সের প্রথম কলাম ছাড়া বাকি কলাম একা ব্যবহার করলে ইনডেক্স ব্যবহৃত হয় না — (ক) তাই full table scan করবে।

অনুশীলন 8.7. Covering index চেনা [এক্সপার্ট]

যদি একটা ইনডেক্স (status, published_at, headline) থাকে, এই কুয়েরি কি covering index সুবিধা পাবে: SELECT headline, published_at FROM articles WHERE status='published'?

সমাধান:

```
-- যদি ইনডেক্স থাকে: INDEX(status, published_at, headline)
SELECT headline, published_at FROM articles
WHERE status = 'published';
```

ব্যাখ্যা: হ্যাঁ — কুয়েরির দরকারি সব কলাম (status ফিল্টারে, published_at ও headline SELECT-এ) ইনডেক্সেই আছে, তাই MySQL মূল টেবিলে না গিয়েই ইনডেক্স থেকে সরাসরি উত্তর দিতে পারবে (EXPLAIN-এ Extra="Using index" দেখাবে) — এটাই সবচেয়ে দ্রুত সম্ভব read।

মডিউল B9

Transaction ও Locking সিনারিও অনুশীলন

বাস্তব রেস-কন্ডিশন সমস্যা সমাধান করা

অনুশীলন 9.1. মৌলিক ট্রানজ্যাকশন লেখা [মাঝারি]

একটা সাবস্ক্রিপশন **cancel** করার সময় **status** আপডেট এবং **subscription_events**-এ একটা ইভেন্ট লগ করা — দুটো একসাথে **atomic** করো।

সমাধান:

```
START TRANSACTION;
UPDATE subscriptions SET status='cancelled', cancelled_at=NOW() WHERE
id=?;
INSERT INTO subscription_events (subscription_id, user_id, event_type,
occurred_at)
VALUES (?, ?, 'cancelled', NOW());
COMMIT;
```

ব্যাখ্যা: দুটো অপারেশন **transaction**-এর ভেতরে — যদি দ্বিতীয় **INSERT** ব্যর্থ হয় (যেমন কানেকশন ছিঁড়ে যায়), প্রথম **UPDATE**-ও **rollback** হয়ে যাবে, "cancelled কিন্তু কোনো ইভেন্ট লগ নেই" এমন অসামঞ্জস্যপূর্ণ অবস্থা তৈরি হবে না।

অনুশীলন 9.2. ওভারস্পেন্ড প্রতিরোধ [কঠিন]

একটা **ad_campaign**-এ নতুন **impression** যোগ করার আগে **budget** চেক করে, **budget**-এর মধ্যে থাকলেই **spent_amount** বাড়াও — **race condition** এড়িয়ে।

সমাধান:

```
START TRANSACTION;
SELECT total_budget, spent_amount FROM ad_campaigns
WHERE id = ? FOR UPDATE;
-- অ্যাপ্লিকেশন কোডে চেক করে: spent_amount + new_cost <= total_budget
UPDATE ad_campaigns
SET spent_amount = spent_amount + ?
WHERE id = ?;
COMMIT;
```

ব্যাখ্যা: **FOR UPDATE** দিয়ে রো লক না করলে দুইটা সমান্তরাল রিকোয়েস্ট একই পুরনো **spent_amount** পড়তে পারে, দুটোই বাজেট-এর-মধ্যে ভেবে আপডেট করতে পারে, ফলে মোট বাজেট ছাড়িয়ে যেতে পারে (**lost update** সমস্যা)।

অনুশীলন 9.3. Optimistic check দিয়ে পাবলিশ [মাঝারি]

একটা **scheduled** আর্টিকেল পাবলিশ করার সময় নিশ্চিত করো যে এটা আগে থেকেই **published** না হয়ে গেছে (**double-publish** আটকানো)।

সমাধান:

```
UPDATE articles
SET status='published', published_at=NOW()
WHERE id = ? AND status = 'scheduled';
-- অ্যাপ্লিকেশন কোডে চেক করে: affected_rows == 1
-- যদি 0 হয়, বুঝবে অন্য কোনো প্রসেস আগেই পাবলিশ করে দিয়েছে
```

ব্যাখ্যা: **WHERE** ক্লজে **status='scheduled'** একটা হালকা **optimistic locking** — যদি ইতিমধ্যে অন্য কোনো **worker** পাবলিশ করে ফেলে (**status** আর **scheduled** নেই), **UPDATE** কিছুই বদলাবে না, কোনো এক্সপ্রিসিট লক না নিয়েও ডাবল-পাবলিশ এড়ানো যায়।

অনুশীলন 9.4. টিকেট বিক্রি — ওভারসেল প্রতিরোধ [কঠিন]

একটা ইভেন্ট টিকেট কেনার সময় `quantity_sold` বাড়ানোর আগে `quantity_available`-এর মধ্যে আছে কিনা চেক করে, তারপরই বাড়ানো।

সমাধান:

```
START TRANSACTION;
SELECT quantity_available, quantity_sold FROM ticket_tiers
WHERE id = ? FOR UPDATE;
-- চেক করো: quantity_sold + requested_qty <= quantity_available
UPDATE ticket_tiers
SET quantity_sold = quantity_sold + ?
WHERE id = ?;
COMMIT;
```

ব্যাখ্যা: একই প্যাটার্ন যেমন `ad_campaigns` বাজেটে — উচ্চ-চাহিদার ইভেন্টে (একই সাথে অনেকে টিকেট কিনতে চেষ্টা করে) `FOR UPDATE` ছাড়া ওভারসেল (`available`-এর বেশি টিকেট বিক্রি) হয়ে যেতে পারে।

অনুশীলন 9.5. Deadlock সিনারিও চেনা [এক্সপার্ট]

দুইটা ট্রানজ্যাকশন একসাথে চললে কেন **deadlock** হতে পারে ব্যাখ্যা করো: ট্রানজ্যাকশন A প্রথমে `row-1` লক করে তারপর `row-2` চায়; ট্রানজ্যাকশন B প্রথমে `row-2` লক করে তারপর `row-1` চায়।

সমাধান:

```
-- Transaction A:
START TRANSACTION;
SELECT * FROM ad_campaigns WHERE id=1 FOR UPDATE;
-- ... কিছুক্ষণ পরে ...
SELECT * FROM ad_campaigns WHERE id=2 FOR UPDATE; -- আটকে যায়

-- Transaction B (একই সময়ে):
START TRANSACTION;
SELECT * FROM ad_campaigns WHERE id=2 FOR UPDATE;
-- ... কিছুক্ষণ পরে ...
SELECT * FROM ad_campaigns WHERE id=1 FOR UPDATE; -- আটকে যায়
```

ব্যাখ্যা: A `row-1` ধরে `row-2`-এর জন্য অপেক্ষা করছে, B `row-2` ধরে `row-1`-এর জন্য অপেক্ষা করছে — দুজনেই একে অপরকে অপেক্ষা করছে, কেউ কখনো এগোতে পারবে না (circular wait)। MySQL এটা স্বয়ংক্রিয়ভাবে সনাক্ত করে একটা ট্রানজ্যাকশনকে জোর করে `ROLLBACK` করে দেয় (deadlock error)। সমাধান: সবসময় একই ক্রমে (যেমন সবসময় ছোট `id` আগে) লক নেওয়ার নিয়ম মেনে চলা।

মডিউল **B10**

এক্সপার্ট — বাস্তব প্রজেক্ট মিশ্র চ্যালেঞ্জ

*একাধিক কনসেপ্ট একসাথে — যেন সত্যিকারের ফিচার বানাচ্ছে*অনুশীলন **10.1.** হোমপেজ ট্রেন্ডিং বিভাগ [এক্সপার্ট]

প্রতিটা ক্যাটাগরির জন্য গত ২৪ ঘণ্টার সবচেয়ে বেশি-পড়া টপ-৩ আর্টিকেল দেখাও, প্রতিটার সাথে ক্যাটাগরির নাম।

সমাধান:

```
SELECT category_name, headline, views_24h, rn
FROM (
  SELECT c.name AS category_name, a.headline,
    COUNT(rh.id) AS views_24h,
    ROW_NUMBER() OVER (
      PARTITION BY a.primary_category_id
      ORDER BY COUNT(rh.id) DESC
    ) AS rn
  FROM articles a
  JOIN categories c ON a.primary_category_id = c.id
  JOIN read_histories rh ON rh.article_id = a.id
  AND rh.read_at >= NOW() - INTERVAL 24 HOUR
  WHERE a.status = 'published'
  GROUP BY a.id, c.name
) ranked
WHERE rn <= 3
ORDER BY category_name, rn;
```

ব্যাখ্যা: JOIN (articles+categories+read_histories) + GROUP BY + COUNT + Window Function (ROW_NUMBER PARTITION BY) সব একসাথে — এটাই একটা বাস্তব "trending by category" API এন্ডপয়েন্টের ভিত্তি কুয়েরি (যদিও প্রোডাকশনে এটা Redis দিয়ে করা হয়, SQL-এ ভিত্তি ধারণা একই)।

অনুশীলন **10.2.** পেওয়াল মিটার সম্পূর্ণ চেক [এক্সপার্ট]

একটা ইউজারের (id=?) এই মাসের মিটার্ড আর্টিকেল-পড়া সংখ্যা বের করো এবং তার প্ল্যানের free_articles_per_month-এর সাথে তুলনা করে বলো সে paywall hit করেছে কিনা।

সমাধান:

```
SELECT
  reads.total_reads,
  p.free_articles_per_month,
  CASE
    WHEN p.free_articles_per_month IS NULL THEN 'unlimited'
    WHEN reads.total_reads >= p.free_articles_per_month THEN 'paywall_hit'
    ELSE 'ok'
  END AS paywall_status
FROM (
  SELECT COUNT(DISTINCT article_id) AS total_reads
  FROM read_histories
  WHERE user_id = ?
    AND read_at >= DATE_FORMAT(NOW(), '%Y-%m-01')
) reads
JOIN subscriptions s ON s.user_id = ? AND s.status = 'active'
JOIN plans p ON s.plan_id = p.id;
```

ব্যাখ্যা: Derived table (COUNT) + JOIN + CASE WHEN — এখানে NULL হ্যান্ডলিং (unlimited প্লানে free_articles_per_month NULL) এবং মেটার্ড গণনা (DISTINCT article_id, এই মাসের রেঞ্জ) দুটোই একসাথে সামলানো হয়েছে।

অনুশীলন 10.3. রেভিনিউ অ্যাট্রিবিউশন ড্যাশবোর্ড [এক্সপার্ট]

গত ৩০ দিনে কোন আর্টিকেল সবচেয়ে বেশি সাবস্ক্রিপশন-রেভিনিউ এনেছে, লেখকের নামসহ, টপ-১০।

সমাধান:

```
SELECT a.headline, u.first_name AS author,
       COUNT(ra.id) AS conversions,
       SUM(ra.assigned_revenue) AS total_revenue
FROM revenue_attribution ra
JOIN articles a ON ra.source_article_id = a.id
JOIN article_author aa ON aa.article_id = a.id AND aa.is_primary = 1
JOIN users u ON aa.user_id = u.id
WHERE ra.attribution_source = 'article_paywall'
      AND ra.created_at >= NOW() - INTERVAL 30 DAY
GROUP BY a.id, u.first_name
ORDER BY total_revenue DESC
LIMIT 10;
```

ব্যাখ্যা: তিন-টেবিল JOIN + WHERE (দুই শর্ত) + GROUP BY + দুই ধরনের aggregate (COUNT, SUM) + ORDER BY + LIMIT — এটা একটা সম্পূর্ণ বিজনেস রিপোর্ট, যেটা এই প্রজেক্টের রেভিনিউ ড্যাশবোর্ডে সরাসরি ব্যবহার হতে পারে।

অনুশীলন 10.4. চার্ন-রিস্ক সেগমেন্টেশন [এক্সপার্ট]

সক্রিয় সাবস্ক্রাইবারদের তাদের গত ৩০ দিনের রিডিং অ্যাক্টিভিটি অনুযায়ী high/medium/low/none রিস্ক-ট্যাগ দাও, এবং প্রতিটা ট্যাগে কতজন আছে গণনা করো।

সমাধান:

```
WITH activity AS (
  SELECT u.id, COUNT(DISTINCT rh.article_id) AS reads_30d
  FROM users u
  LEFT JOIN read_histories rh ON rh.user_id = u.id
  AND rh.read_at >= NOW() - INTERVAL 30 DAY
  WHERE u.subscription_status = 'active'
  GROUP BY u.id
),
tagged AS (
  SELECT id,
  CASE
    WHEN reads_30d = 0 THEN 'high_risk'
    WHEN reads_30d < 3 THEN 'medium_risk'
    WHEN reads_30d < 10 THEN 'low_risk'
    ELSE 'no_risk'
  END AS risk_tag
  FROM activity
)
SELECT risk_tag, COUNT(*) AS user_count
FROM tagged
GROUP BY risk_tag
ORDER BY user_count DESC;
```

ব্যাখ্যা: দুইটা CTE চেইন করা হয়েছে (activity → tagged), LEFT JOIN (যাতে ০-পড়া ইউজারও বাদ না পড়ে), CASE WHEN দিয়ে সেগমেন্ট, তারপর সেই সেগমেন্টের উপর আরেকবার GROUP BY — এটা retention_offers (মডিউল-১০) কাকে দেখাবে তা ঠিক করার আসল লজিকের ভিত্তি।

অনুশীলন 10.5. এডিটোরিয়াল প্রোডাক্টিভিটি রিপোর্ট [এক্সপার্ট]

প্রতিটা ডেস্কের (editorial_desks) জন্য এই মাসে কতগুলো অ্যাসাইনমেন্ট সম্পন্ন হয়েছে (completed) এবং গড়ে কত দিনে (assigned থেকে completed_at পর্যন্ত) — ডেস্কের নামসহ।

সমাধান:

```
SELECT ed.name AS desk_name,
       COUNT(*) AS completed_assignments,
       AVG(DATEDIFF(ea.completed_at, ea.created_at)) AS avg_days_to_complete
FROM editorial_assignments ea
JOIN editorial_desks ed ON ea.desk_id = ed.id
WHERE ea.status = 'completed'
      AND ea.completed_at >= DATE_FORMAT(NOW(), '%Y-%m-01')
GROUP BY ed.id, ed.name
ORDER BY avg_days_to_complete ASC;
```

ব্যাখ্যা: JOIN + WHERE (স্ট্যাটাস ও সময়সীমা) + AVG(DATEDIFF(...)) — দুটো তারিখ কলামের পার্থক্যের গড় বের করা একটা সাধারণ কিন্তু গুরুত্বপূর্ণ প্যাটার্ন, "average turnaround time" জাতীয় যেকোনো অপারেশনাল মেট্রিকে ব্যবহার হয়।

অনুশীলন 10.6. A/B টেস্ট রেজাল্ট বিশ্লেষণ [\[এক্সপার্ট\]](#)

একটা এক্সপেরিমেন্টের (id=?) প্রতিটা ভ্যারিয়েন্টে কতজন ইউজার assign হয়েছে, এবং তাদের মধ্যে কতজন সাবসক্রাইব করেছে (conversion rate সহ)।

সমাধান:

```
SELECT aea.variant,
       COUNT(DISTINCT aea.user_id) AS assigned_users,
       COUNT(DISTINCT s.user_id) AS converted_users,
       ROUND(COUNT(DISTINCT s.user_id) / COUNT(DISTINCT aea.user_id) * 100, 2)
       AS conversion_pct
FROM ab_experiment_assignments aea
LEFT JOIN subscriptions s ON s.user_id = aea.user_id
      AND s.created_at >= aea.assigned_at
WHERE aea.experiment_id = ?
GROUP BY aea.variant;
```

ব্যাখ্যা: LEFT JOIN জরুরি (যারা কনভার্ট করেনি তারাও গণনায় থাকতে হবে assigned_users-এর জন্য) + শর্তসহ JOIN (s.created_at >= aea.assigned_at, যাতে assignment-এর আগের পুরনো সাবসক্রিপশন ভুলভাবে "conversion" হিসেবে না গণ্য হয়) + conversion rate গণনা — এটাই A/B টেস্ট বিশ্লেষণের মূল কুয়েরি প্যাটার্ন।

অনুশীলন 10.7. সম্পূর্ণ ফ্রিল্যান্সার পেমেন্ট রিপোর্ট [\[এক্সপার্ট\]](#)

একজন ফ্রিল্যান্সারের (user_id=?) এই মাসের সব ইনভয়েন্স আইটেম, প্রতিটা ইনভয়েন্সের সাবটোটাল-সহ রানিং টোটাল, এবং সব মিলিয়ে সর্বমোট কত টাকা তার একসাথে দেখাও।

সমাধান:

```
SELECT fi.invoice_number, ii.description, ii.amount,
       SUM(ii.amount) OVER (PARTITION BY fi.id ORDER BY ii.id) AS
       invoice_running_total,
       SUM(ii.amount) OVER () AS grand_total
FROM freelancer_invoices fi
JOIN invoice_items ii ON ii.invoice_id = fi.id
WHERE fi.user_id = ?
      AND fi.period_start >= DATE_FORMAT(NOW(), '%Y-%m-01')
ORDER BY fi.invoice_number, ii.id;
```

ব্যাখ্যা: দুইটা ভিন্ন window function একসাথে — একটা PARTITION BY fi.id দিয়ে প্রতি-ইনভয়েন্স রানিং টোটাল, আরেকটা কোনো PARTITION BY ছাড়াই (পুরো রেজাল্ট-সেটের) সর্বমোট। এটা দেখায় কীভাবে একই SELECT-এ ভিন্ন ভিন্ন "স্কোপের" window function বেশানো যায়।

চূড়ান্ত সিদ্ধান্ত-ইনডেক্স — কোন কনসেপ্ট কোথায়

এই টেবিল পুরো নোটের একটা কম্প্যাক্ট ম্যাপ — কোন অধ্যায় কোন অনুশীলন শুরুর সাথে সম্পর্কিত, আর প্রতিটা কনসেপ্ট প্রজেক্টের কোথায় সরাসরি ব্যবহার হবে।

অধ্যায় (Part A)	সংশ্লিষ্ট অনুশীলন (Part B)	প্রজেক্টে সরাসরি ব্যবহার
১-৩ — বেসিক, ডেটা টাইপ	B1	প্রতিটা লিস্টিং API, ফর্ম ভ্যালিডেশন
৪-৫ — JOIN, Self-JOIN	B2	নেস্টেড রেসপন্স, categories/comments হায়াারার্কি
৬-৭ — Aggregation, HAVING	B3	প্রায় সব ড্যাশবোর্ড ও রিপোর্ট
৮-৯ — Subquery, CTE	B4	জটিল paywall rules, রিপোর্ট বিল্ডিং
১০ — Window Functions	B5	ট্রেন্ডিং, revenue ranking, রা‌নিং টোটাল
১১-১২ — Set Ops, CASE WHEN	B6	সেগমেন্টেশন, চার্ন-রিস্ক ট্যাগিং
১৩-১৫ — String, Date, NULL	B7-B8	প্রায় প্রতিটা কুয়েরির ছোট অংশ হিসেবে
১৬ — Index ও EXPLAIN	B8	পারফরম্যান্স টিউনিং, প্রতিটা "listing" কুয়েরি
১৭ — N+1	(কনসেপ্টচুয়াল)	সব nested API রেসপন্স
১৮ — Full-text, JSON	(কনসেপ্টচুয়াল)	সার্চ ফলব্যাক, plans/rules/targeting কলাম
১৯ — Partitioning	(কনসেপ্টচুয়াল)	behavioral_metrics, read_histories, activity_logs
২০ — Pagination	(কনসেপ্টচুয়াল)	হোমপেজ ফিড, ক্যাটাগরি পেজ
২১ — Transaction, Locking	B9	পাবলিশ, পেমেন্ট, বাজেট/টিকেট বিক্রি
২২-২৩ — Normalization, View/Trigger	(কনসেপ্টচুয়াল)	পুরো স্কিমা ডিজাইনের ভিত্তি
সব একসাথে	B10	বাস্তব ফিচার — ট্রেন্ডিং, পেওয়াল, রেভিনিউ, চার্ন, A/B টেস্ট

টপ-ক্রাস হওয়ার পথে পরবর্তী ধাপ

- এই ১২০+ অনুশীলন একবার শেষ করার পর, আবার প্রথম থেকে শুরু করো — এবার সমাধান না দেখে, শুধু প্রশ্ন পড়ে নিজে লেখো। দ্বিতীয়বার অনেক দ্রুত এবং সহজ লাগবে।
- একটা লোকাল **MySQL** ডেটাবেজে এই প্রজেক্টের স্কিমা (কমপক্ষে ১০-১৫টা মূল টেবিল) বানিয়ে ফেক ডেটা (**Faker** লাইব্রেরি দিয়ে) ভরে ফেলো — তারপর প্রতিটা অনুশীলন সত্যিকারের ডেটার উপর রান করে দেখো, ফলাফল যা আশা করেছিলে তার সাথে মেলাও।
- যখনই একটা নতুন ফিচারের জন্য কুয়েরি লিখতে বসবে, নিজেকে জিজ্ঞেস করো: এটা কি সাধারণ **SELECT** (অধ্যায় ২), নাকি একাধিক টেবিল লাগবে (৪), নাকি গণনা/গ্রুপিং (৬-৭), নাকি "প্রতি-গ্রুপে" কিছু বের করতে হবে (১০ — **Window Function**)? এই প্রশ্নগুলো ধাপে ধাপে করলে সঠিক প্যাটার্ন বেছে নেওয়া সহজ হয়ে যাবে।
- EXPLAIN** ব্যবহার করা অভ্যাসে পরিণত করো — প্রতিটা নতুন কুয়েরি লেখার পর একবার **EXPLAIN** চালিয়ে দেখো **type** ও **key** কলাম কী বলছে।
- এই প্রজেক্ট ডেভেলপ করার সময় প্রতিটা মডিউলের নোট (আগে বানানো "মডিউল-ওয়াইজ রিভিশন নোট") আর এই কুয়েরি নোট পাশাপাশি রেখে কাজ করো — "কেন এই টেবিল এভাবে ডিজাইন হলো" (মডিউল নোট) আর "কীভাবে এটা কুয়েরি করব" (এই নোট) একসাথে মিললেই পুরো ছবিটা স্পষ্ট হবে।